Static Timing Analysis

ChipRun // Digital-design

Episode 02 — STA

Phase 01

It's been a long run,

you are here

more.pending

Verilog
COMPLETE

STA
RUNNING

Power
QUEUED

CDC
QUEUED

RDC
QUEUED

DFT
QUEUED

Synth
QUEUED

...

But We haven't $arrived yet.

GitHub
code examples

Drive
full files and slides

LinkedIn
new episode alerts

Topics

Topics

1. Introduction
2. Interconnect and Cell Delays
3. Timing Paths & Basic Concepts
4. Timing Analysis Fundamentals
5. Fixing Timing Violations
6. Timing Constraints & SDC
7. Advanced Timing Concepts
8. Signoff & Corners
9. References

Introduction

Introduction

Why STA is Critical

Why STA is Critical for Modern Chip Design:

- Ensures design meets timing requirements at all operating conditions
- Verifies signal propagation within clock period constraints
- Catches timing violations before expensive silicon fabrication
- Required for signoff at every semiconductor company
- Dynamic simulation alone cannot verify all timing scenarios

Without proper STA:

- Chips may fail at certain voltage/temperature combinations
- Setup/hold violations cause metastability and functional failures
- Performance targets may not be met in production

Introduction

STA vs Dynamic Simulation

Static Timing Analysis (STA):

- Analyzes all possible timing paths statically
- Fast - does not require simulation vectors
- Comprehensive - covers every path in the design
- Checks timing at worst-case conditions
- Cannot verify functional correctness

Dynamic Simulation:

- Simulates design with test vectors
- Slower - needs comprehensive stimulus
- May miss some timing scenarios
- Verifies functional operation
- Used with STA for complete verification

Introduction

When to Use STA

Use STA When:

- Verifying setup/hold time requirements
- Checking timing closure after synthesis/place-and-route
- Analyzing clock domain crossings (CDC)
- Signoff for tape-out (manufacturing)
- Optimizing design performance

STA Complements:

- Functional verification (testbench simulation)
- Formal verification (equivalence checking)
- Power analysis (dynamic/leakage power)
- Physical verification (DRC/LVS)

Introduction

Industry Requirement

Industry Standard Requirements:

- Every semiconductor company requires STA signoff
- Foundries (TSMC, Samsung, Intel) mandate timing verification
- IP vendors must provide timing-verified designs
- Fabless companies depend on STA for product reliability

Career Impact:

- STA skills are essential for digital design roles
- Timing engineers are highly valued in the industry
- Understanding STA is key to becoming a signoff engineer
- Interview questions frequently test STA fundamentals

Introduction

Industry Requirement

Industry Standard Requirements:

- Every semiconductor company requires STA signoff
- Foundries (TSMC, Samsung, Intel) mandate timing verification
- IP vendors must provide timing-verified designs
- Fabless companies depend on STA for product reliability

Career Impact:

- STA skills are essential for digital design roles
- Timing engineers are highly valued in the industry
- Understanding STA is key to becoming a signoff engineer
- Interview questions frequently test STA fundamentals

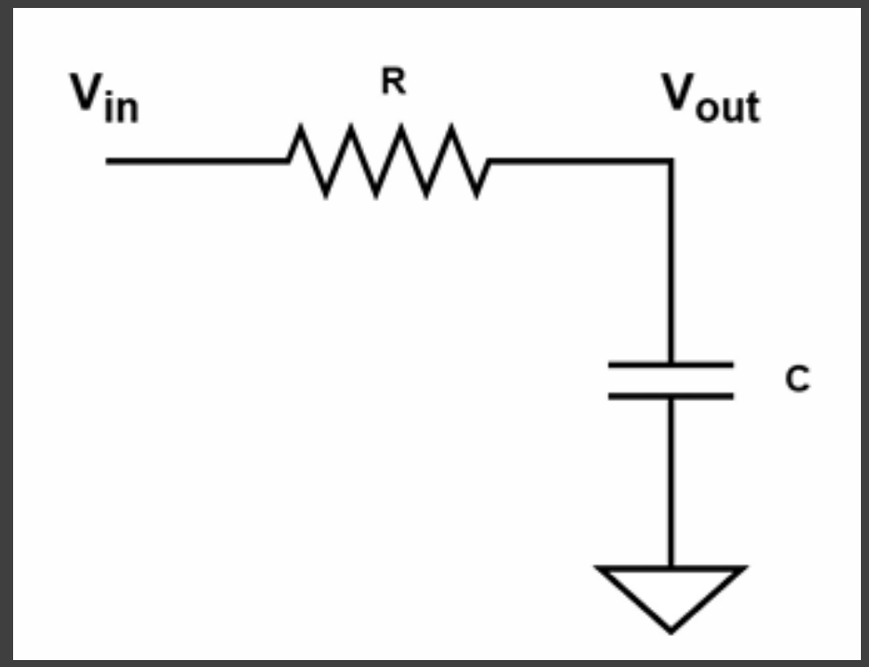
Interconnect and Cell Delays

Interconnect and Cell Delays

RC Delay

- Signals take time to reach the intended value; and this is because of the interconnection delay.

- We can assume that each wire is an RC circuit where the capacitance needs to charge or deplete the voltage for the signal to achieve the wanted value.



- The voltage over the capacitor is governed by the following equation:

$$V_{out} = V_{in} * \left(1 - e^{-\frac{t}{RC}}\right)$$

Interconnect and Cell Delays

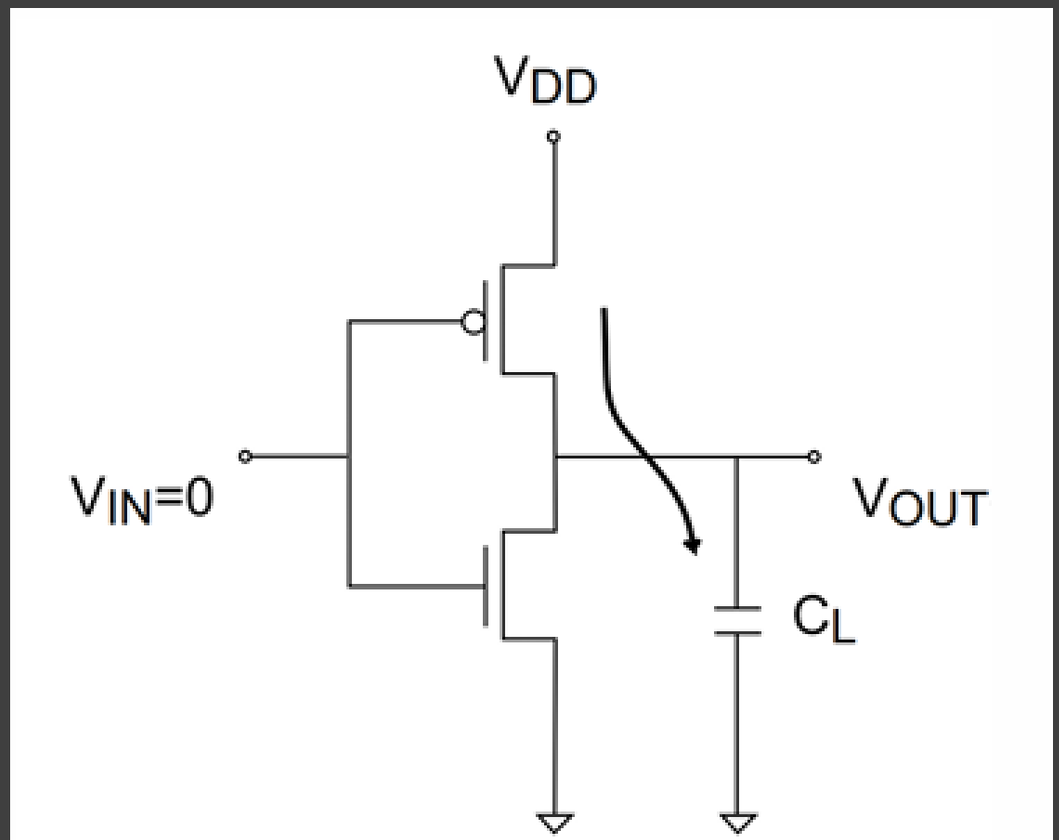
RC Delay

- We say the signal has propagated through the circuit if the capacitor voltage reached 50% of the supply voltage.

- So, by substituting V_{out} with $\frac{V_{in}}{2}$ in the previous equation we get that t = 0.69RC.

- Not only the delay comes from wires, but it can also come from the transistors in the path.

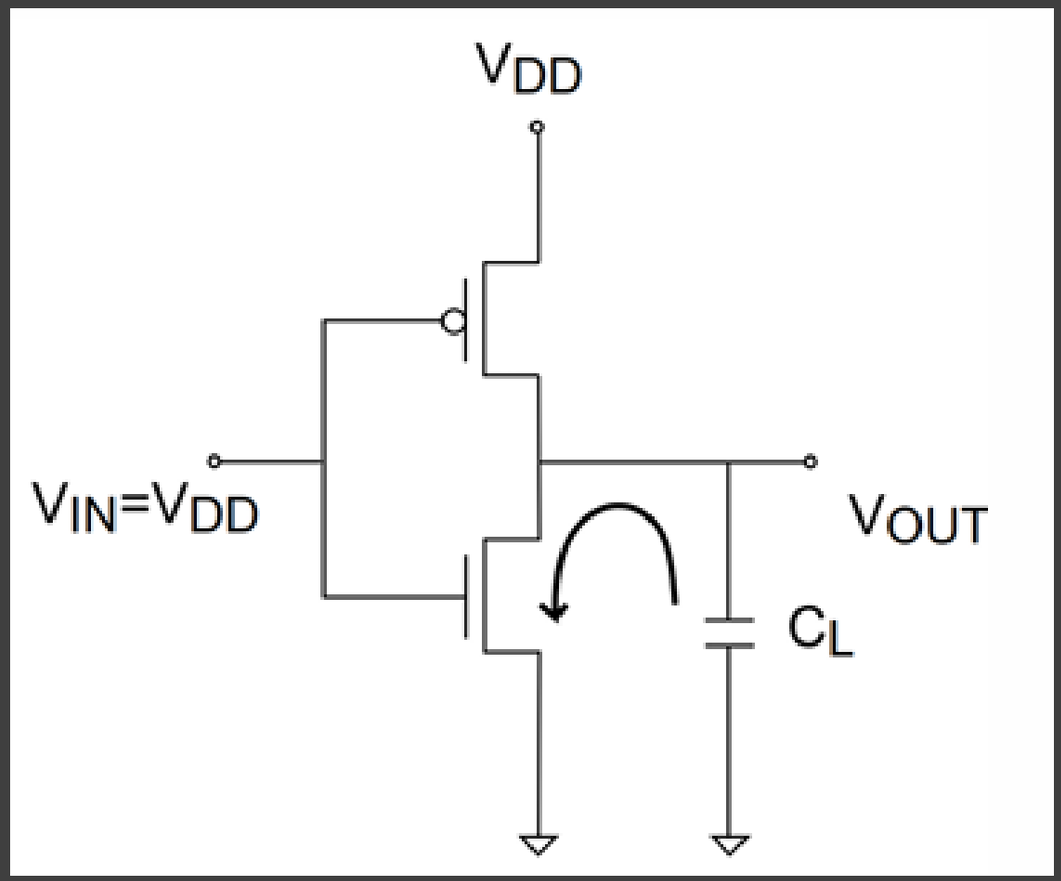
- When the output gets charged from 0 to 1 the current flows from the V_{DD} into the output node and in this path the $R = R_{PMOS}$.
- So, the t_{LH} = 0.69 $* R_{PMOS} * C_L$



Interconnect and Cell Delays

RC Delay

- When the output gets deplete from 1 to 0 the current flows from the output node to the ground and in this path the $R = R_{NMOS}$..
- So, the t_{HL} = 0.69 $* R_{NMOS} * C_L$



- In both cases,

$$R = \frac{V_{DD}}{\frac{W}{2L} * \mu * C_{ox} * (V_{DD} - V_{Th})^2}$$

So, the delay depends on:

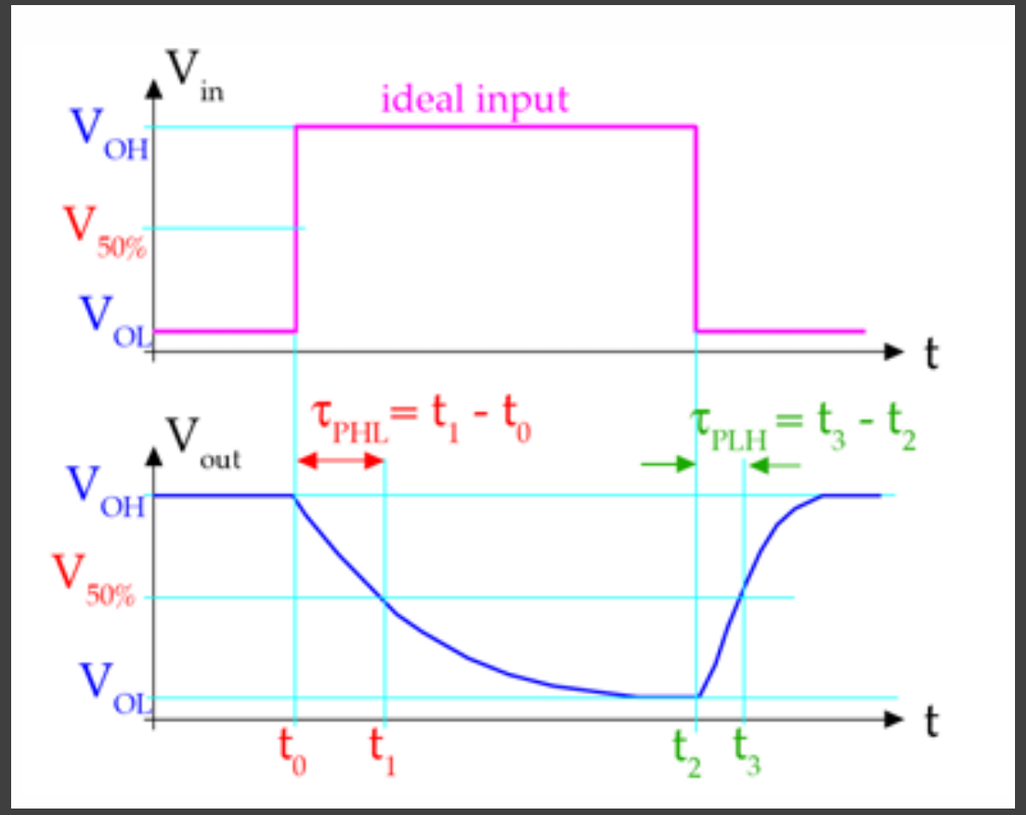
- V_{DD}
- C_L
- V_{TH}
- Transistor width
- Transistor length

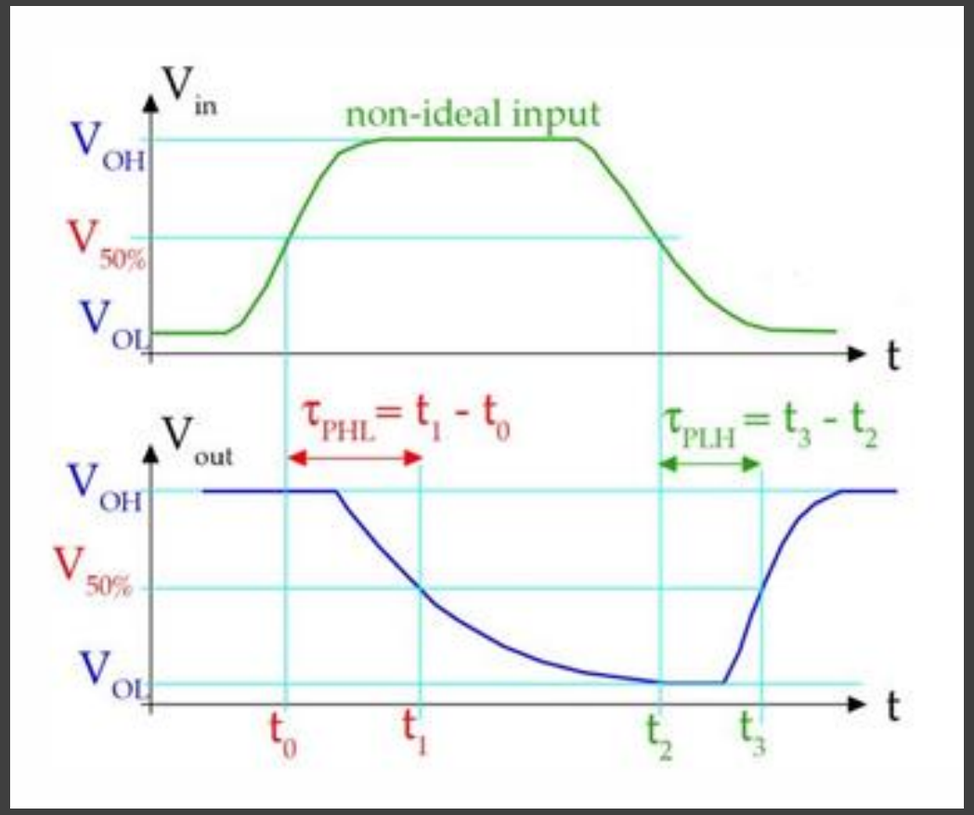
Interconnect and Cell Delays

Transition Time

- For non ideal input, the propagation delay is defined to be the difference between the time when Vin reaches 50% and when $Vout$ reaches 50%

- The slower the input transition the slower the propagation delay





Timing Paths & Basic Concepts

Timing Paths & Basic Concepts

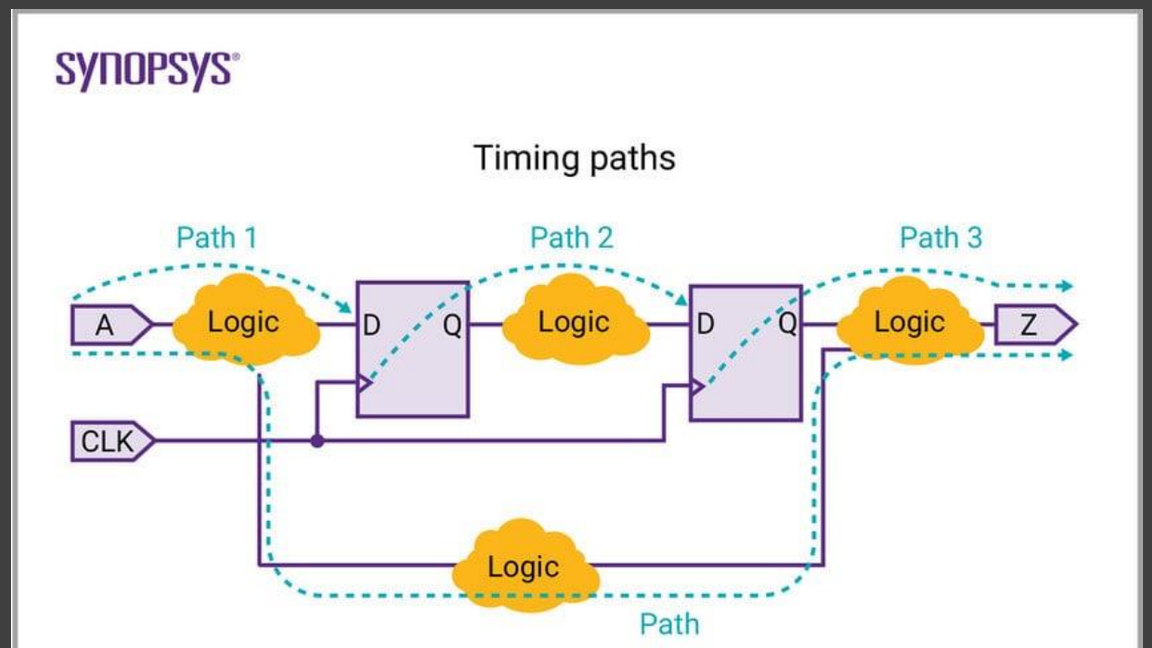
What is a Timing Path?

Timing Path Definition:

- A timing path is a route along which data propagates From a start point to an end point.
- **Start points:** input ports, clock pins of sequential elements
- **End points:** output ports, data pins of sequential elements
- STA analyzes every possible timing path in the design

Path Components:

- Launch flip-flop (or input port)
- Combinational logic network
- Capture flip-flop (or output port)



In the example, each logic cloud represents a combinational logic network. Each path starts at a data launch point, passes through some combinational logic, and ends at a data capture point.

Path	Startpoint	Endpoint
Path 1	Input port	Data input of a sequential element
Path 2	Clock pin of a sequential element	Data input of a sequential element
Path 3	Clock pin of a sequential element	Output port
Path 4	Input port	Output port

Timing Paths & Basic Concepts

Types of Timing Paths

- **Register-to-Register (Reg2Reg):**
  Data path from one flip-flop to another
  Most common timing path in synchronous designs

- **Input-to-Register (In2Reg):**
  From external input to internal flip-flop
  Checks input arrival relative to capture clock

- **Register-to-Output (Reg2Out):**
  From internal flip-flop to external output
  Checks output availability relative to clock

- **Input-to-Output (In2Out):**
  Purely combinational path through the chip

- Most of the STA Analysis happens for the Reg2Reg timing paths

Timing Paths & Basic Concepts

Launch and Capture Concepts

Launch Edge:

- The active clock edge that launches data from the launch flip-flop

Capture Edge:

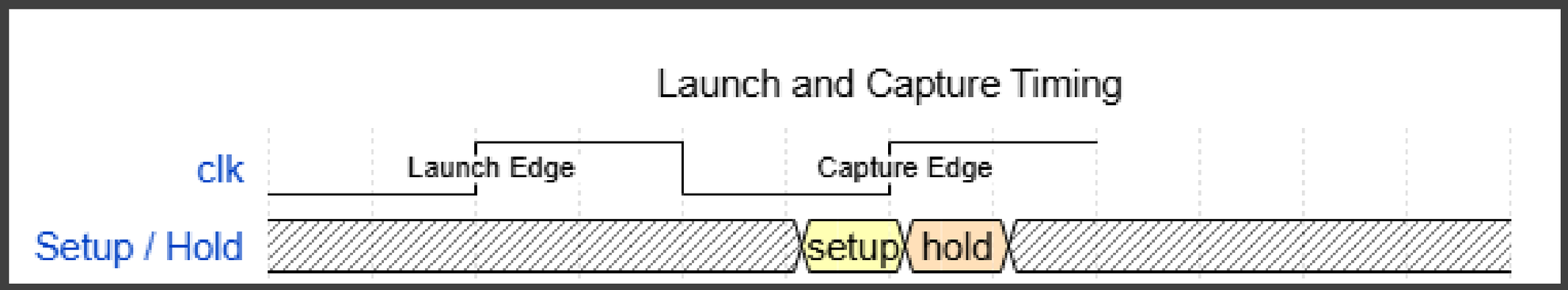
- The active clock edge that captures data at the capture flip-flop
- Data must be stable before this edge (setup time)
- Data must remain stable after this edge (hold time)

Setup Check:

Data must arrive before setup time before capture edge

Hold Check:

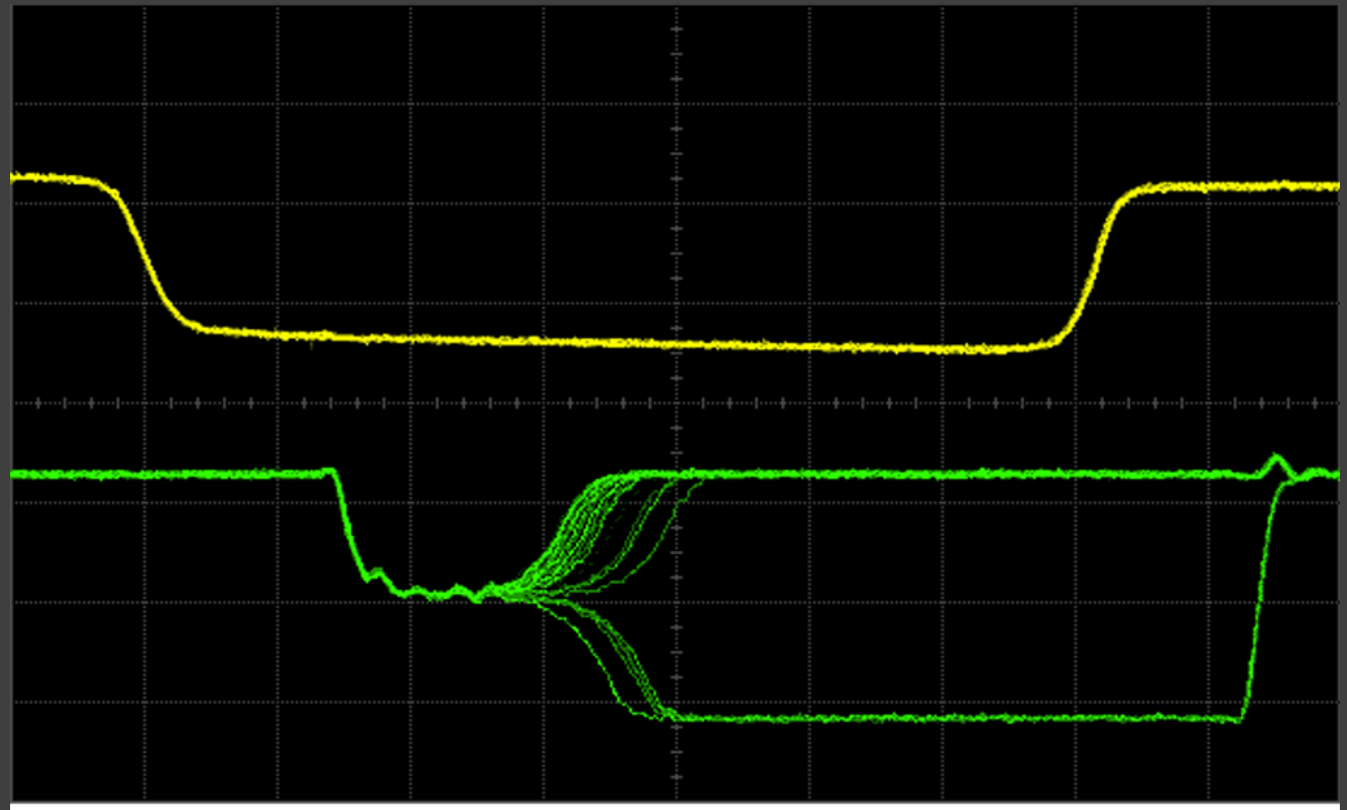
Data must remain stable after hold time after capture edge



Timing Paths & Basic Concepts

Metastability

- if the data changes right after or before the capture edge so it caused setup or hold time violations metastability will happen.
- Metastability is the state where is output of the FF is unknown.
- It becomes a value between 1 and 0 “metastable” before it settles to 0 or 1.
- This is why we need to check for any setup/hold violations in out design to avoid any metastability.



Picture taken from W. J. Dally, Lecture notes for EE108A, Lecture 13: Metastability and Synchronization Failure 11/9/2005.

Timing Paths & Basic Concepts

Timing annotations

- **Propagation delay (t_{pcq})**

  Max time for FF output (Q) to become stable

- **Contamination delay (t_{ccq})**

  Min time before Q starts to change

- **Combinational delay (t_{pd})**

  Longest input→output delay through logic

- **Combinational delay (t_{cd})**

  Shortest input→output delay through logic

Timing Analysis Fundamentals

Timing Analysis Fundamentals

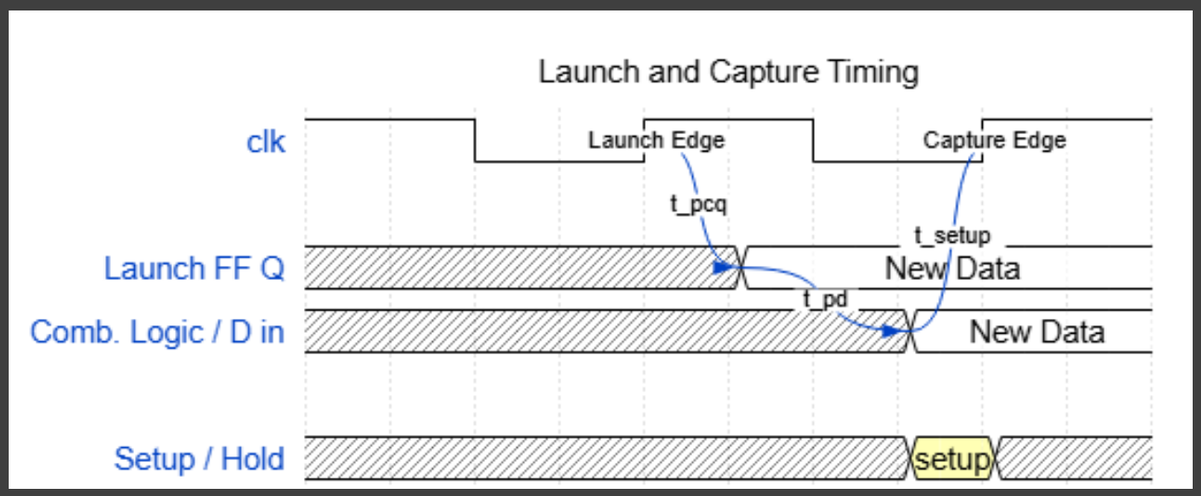
Setup Time Analysis

Setup Time Definition:

- Minimum time **BEFORE** the clock edge that data must be stable
- Ensures data is reliably captured by the flip-flop
- Specified in standard cell library

- For the data to reach from the clock pin of the launch FF to the input pin in the capture FF it has to go through:
  1. Clock to q duration (T_{cq})
  2. Combinational delay

- And it has to reach before the capture edge with at least the setup time value



Timing Analysis Fundamentals

Setup Time Analysis

- **Setup Equation:**

$$T_{clk} \geq T_{pcq} + T_{pd} + T_{setup}$$

- **Setup Slack:**

  Slack = Required Time - Arrival Time

  Positive slack: timing met

  Negative slack: violation

- **Common Interview Question:**

  What happens if setup time is violated?

  → *Metastability, wrong data captured*

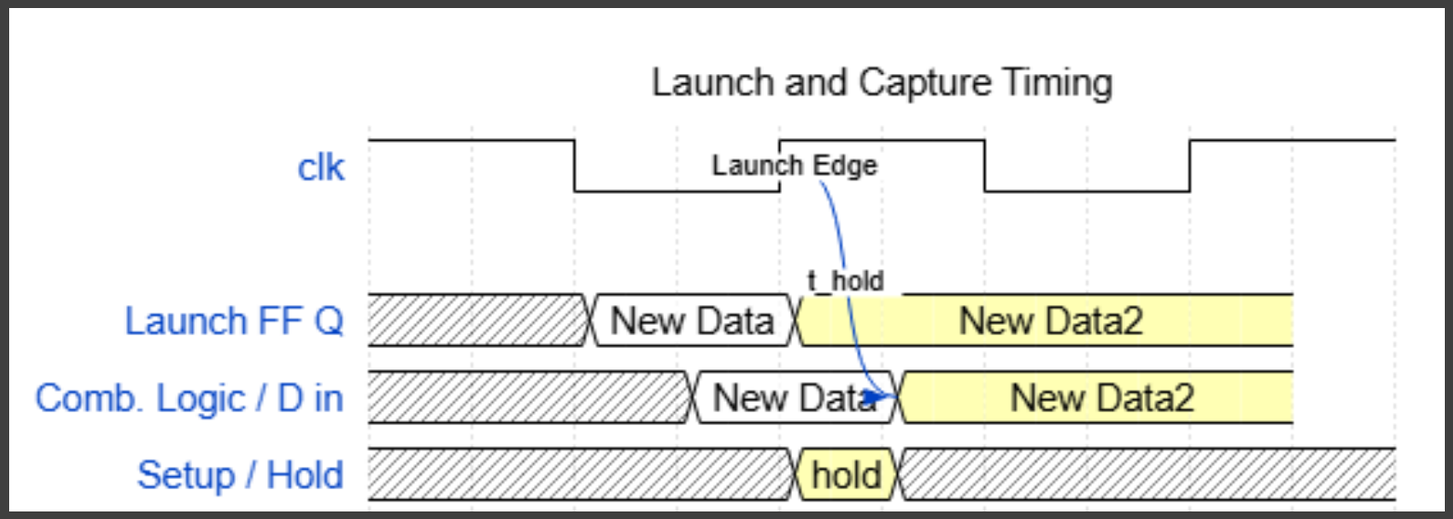
Timing Analysis Fundamentals

Hold Time Analysis

Hold Time Definition:

- Minimum time **AFTER** the clock edge that data must remain stable
- Prevents data from changing too quickly after capture
- Critical for correct operation

- The data should have delay after the launch edge with at least the hold time.



Timing Analysis Fundamentals

Hold Time Analysis

Hold Equation:

$$T_{ccq} + T_{cd} \geq T_{hold}$$

Hold Slack:

Slack = Arrival Time - Required Time

Positive slack: timing met

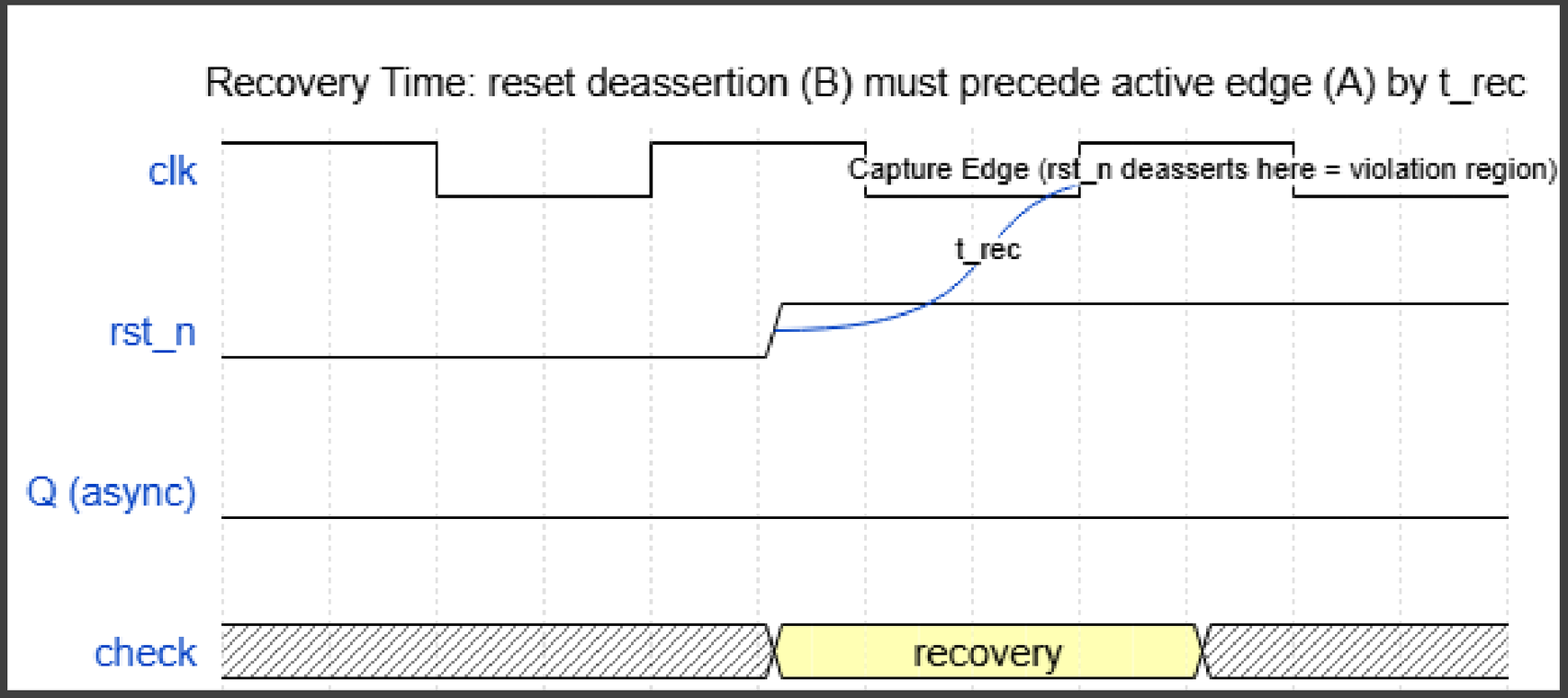
Negative slack: violation

Timing Analysis Fundamentals

Recovery Time

Recovery Time:

- Similar to setup, but for asynchronous signals (reset/clear)
- Minimum time between de-assertion of reset and active clock edge
- Ensures flip-flop can properly capture data on next cycle

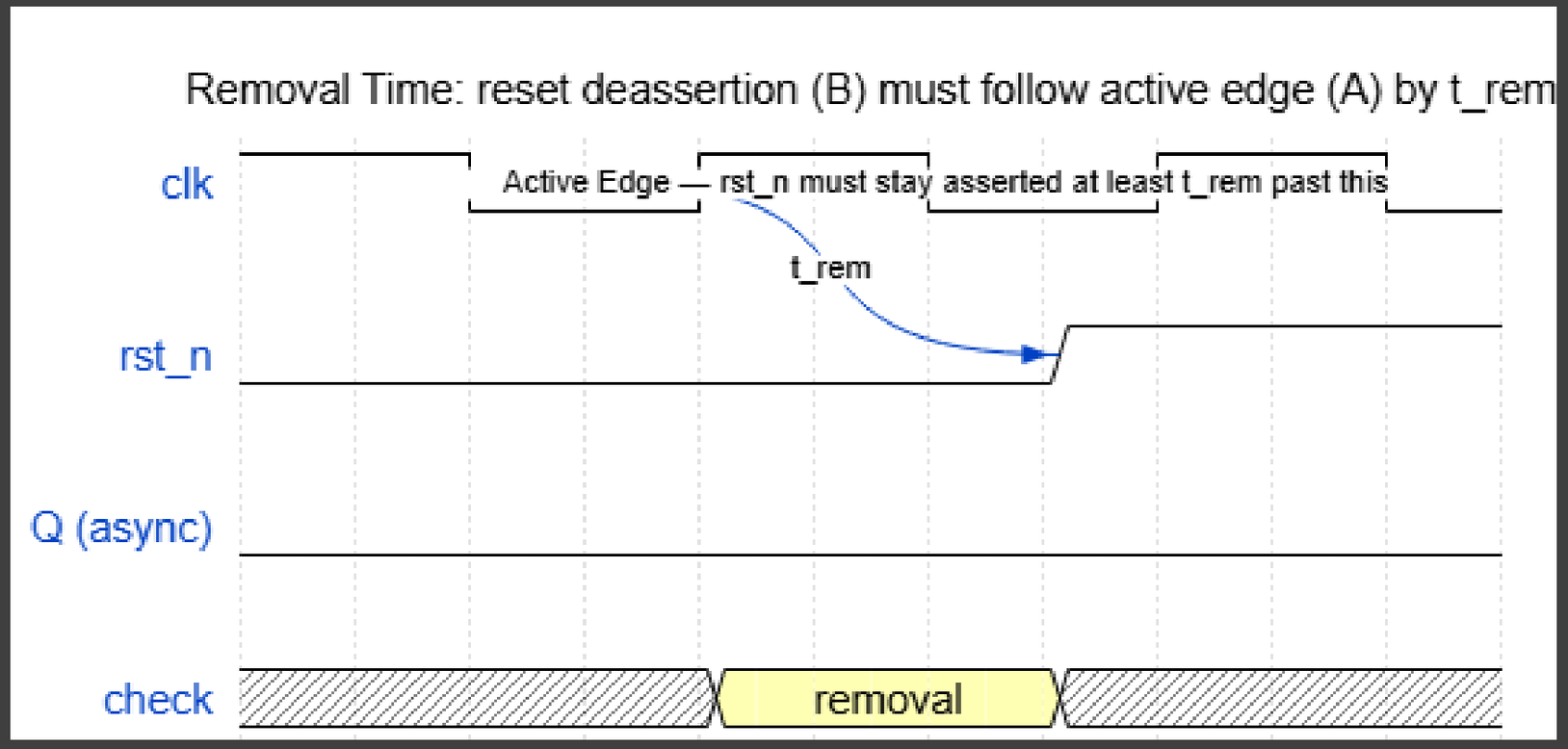


Timing Analysis Fundamentals

Removal Time

Removal Time:

- Similar to hold, but for asynchronous signals
- Minimum time that reset/clear must remain stable after clock edge
- Prevents accidental reset during normal operation



Timing Analysis Fundamentals

Clock Skew

Clock Skew Definition:

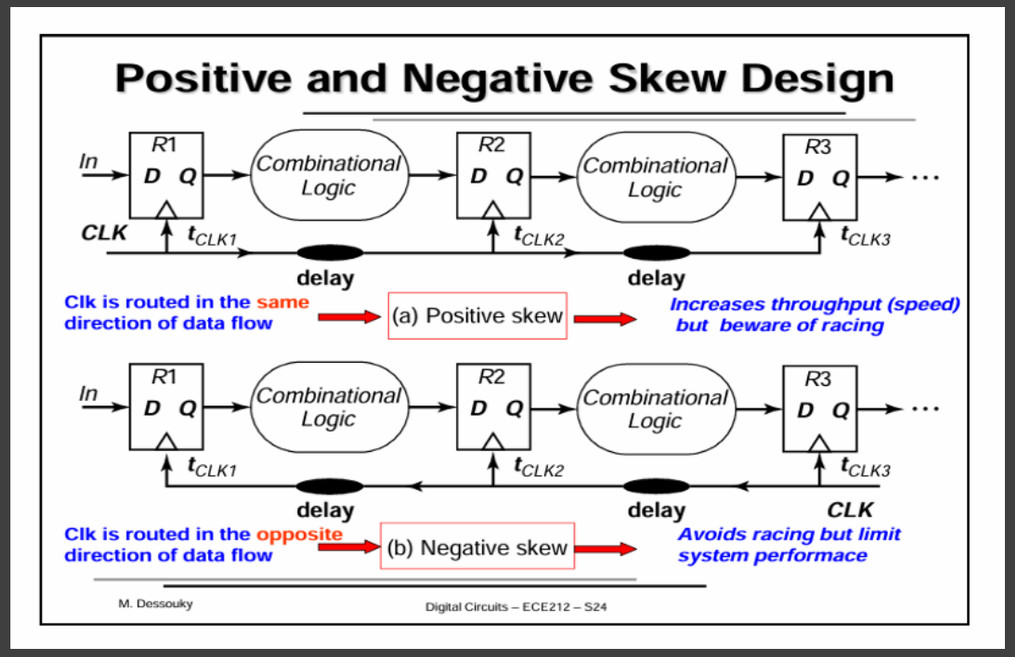
- Difference in clock arrival times at different flip-flops
- Can be positive or negative
- Caused by clock tree imbalances, process variations

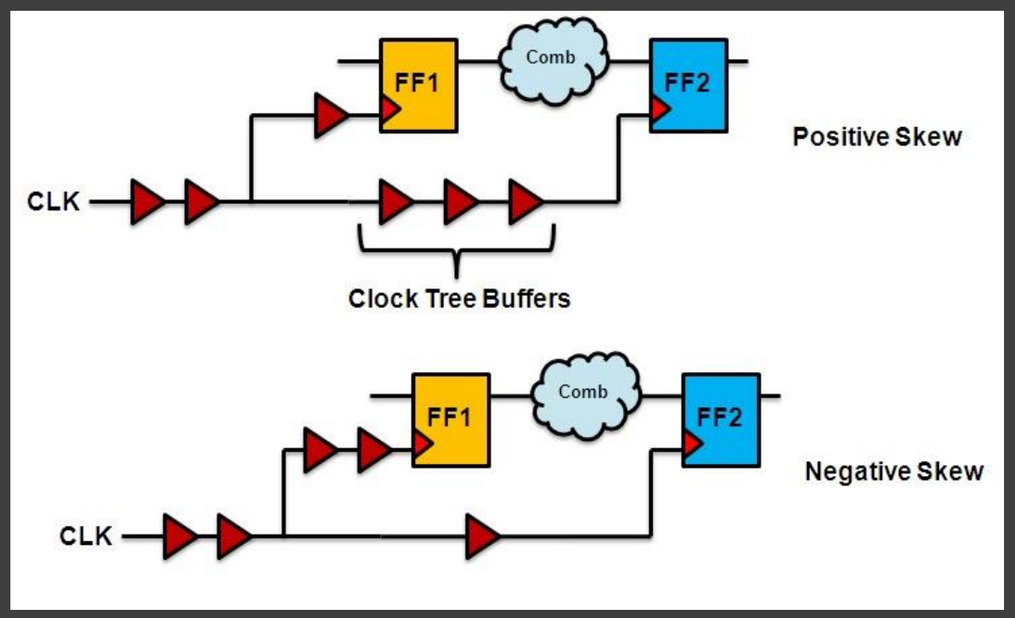
Types of Clock Skew:

- **Positive Skew:** Capture clock arrives AFTER launch clock

  (beneficial for setup, harmful for hold)
- **Negative Skew:** Capture clock arrives BEFORE launch clock

  (harmful for setup, beneficial for hold)
- **Global Skew:** Difference between clock sources
- **Local Skew:** Difference at sequential elements





Timing Analysis Fundamentals

Clock Uncertainty / jitter

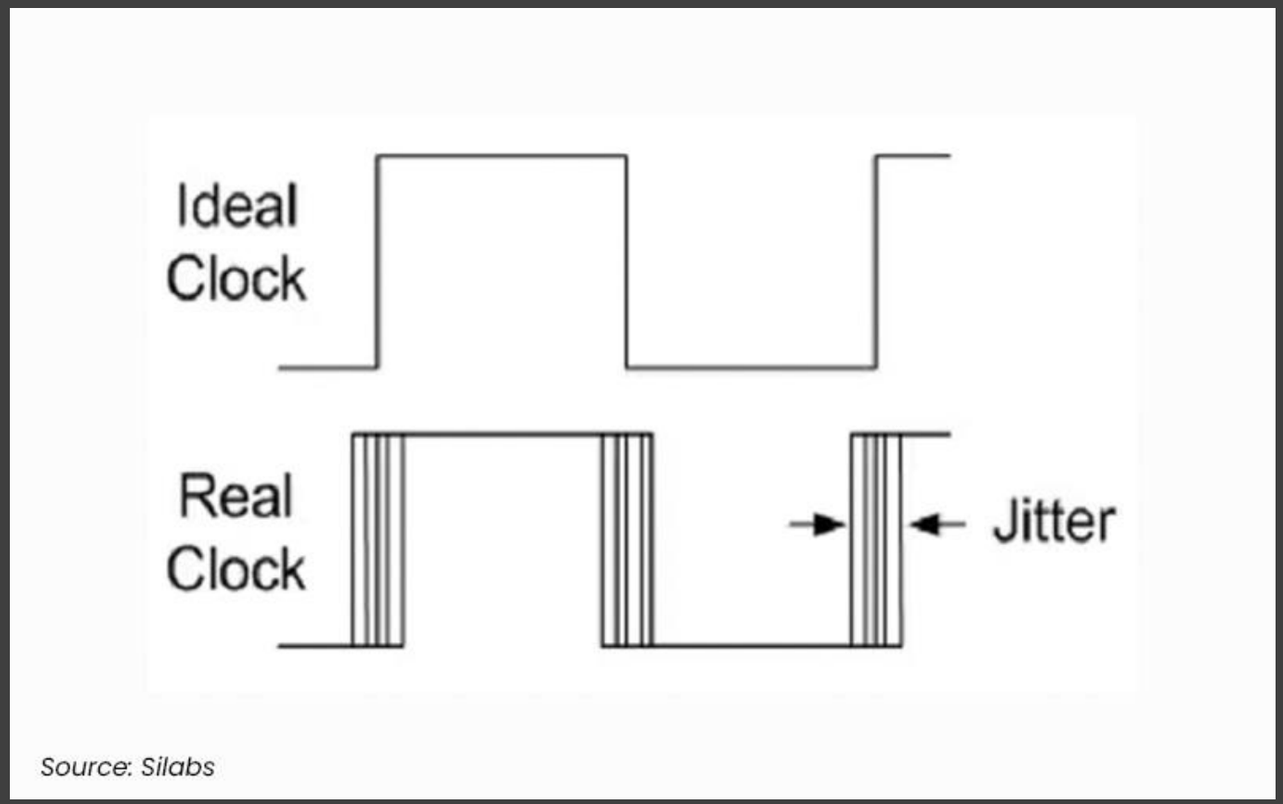
Jitter Definition:

- Cycle-to-cycle variation in clock period, caused by noise in the clock source (PLL/DLL) or power supply
- Unlike skew (a spatial difference between two points in the clock tree), jitter is a **temporal** variation at a *single* point, edge to edge

Caused by:

- PLL/DLL phase noise
- power supply noise (SSN)
- thermal noise
- crosstalk on the clock net

- This change the effective clock Period, which effects the setup and Hold analysis.



Source: Silabs

Timing Analysis Fundamentals

Setup and Hold equations

- Due to the uncertainties and the skew effects, the setup and hold equations will need to change to takes their effects into consideration.

Setup Equation

$$T_{clk} \geq T_{pcq} + T_{pd} + T_{setup} + T_{jitter} - T_{Skew}$$

Hold Equation

$$T_{ccq} + T_{cd} - T_{jitter} - T_{skew} \geq T_{hold}$$

Jitter will be taken as positive value in the setup analysis to design at the worst case.
Skew will be taken as its sign “positive for positive skew and negative for negative skew”

Fixing Timing Violations

Fixing Timing Violations

Introduction

- In the previous sections we defined setup and hold checks and derived their equations, including the effects of skew, jitter and uncertainty.

- In this section we go through the practical techniques used to fix setup and hold violations across the digital design flow.

- The checks covered in this section are:
  - Setup timing
  - Hold timing

Fixing Timing Violations

Overview of the Digital VLSI Flow

The Digital VLSI Flow:

- Specifications & Architecture: define functionality, performance, power and cost, then the blocks, operating voltage and target clock frequency
- RTL Design & Simulation: implement and functionally verify the architecture
- Synthesis: translate RTL into logic gates and standard cells
- Place & Route (PNR): floorplan, power grid, placement, clock tree synthesis, routing, timing closure, then final DRC/LVS/EMIR checks

Where Violation Fixing Fits In:

- Each stage offers its own set of timing optimizations
- Earlier stages fix large violations that are difficult or impossible to fix later
- As the flow progresses, fixes become smaller, trickier, and more manual

Fixing Timing Violations

Setup Violation Fix Techniques — Overview

Early-stage techniques (Architecture, RTL, Synthesis):

- Reducing clock frequency, smaller tech node, higher supply voltage
- Changing the architecture or optimizing the RTL structure
- Pipelining, multi-cycle paths (MCP), retiming
- Optimizing synthesis effort/flattening, false path constraints

Late-stage techniques (Floorplan, PNR, ECO):

- Optimizing the floorplan, wire delay and power grid
- Upsizing, MTCMOS (Vt swap), increasing driving strength
- Breaking up long nets, register duplication
- Reducing crosstalk, adjusting local skew

Fixing Timing Violations

Sol. 1 — Reducing the Clock Frequency

- The simplest solution: lowering the clock frequency (increasing the period) adds time to the required capture side of the equation
- This directly degrades performance — data rate, CPU speed, operations per second, etc.
- This decision belongs to the architecture team, not RTL or PNR engineers, since it affects the whole chip
- Sometimes it isn't acceptable at all because a product standard mandates a specific data rate

Fixing Timing Violations

Sol. 2 & 3 — Smaller Node & Higher Voltage

Moving to a smaller technology node:

- A shorter transistor length reduces gate delay
- But smaller nodes mean higher fabrication cost and longer design cycles — more challenging on-chip variation (OCV) and tighter design rules
- The target node is decided very early via quick hand calculations (average cell delay × average path length) or a trial synthesis project

Increasing the supply voltage:

- Higher VDD reduces propagation delay ($t_{prop} \propto VDD / (VDD - Vth)^2$)
- But dynamic power grows quadratically with VDD, So, there is a tradeoff between this solution and power.
- Higher voltage can be applied selectively to performance-critical blocks, leaving the rest at lower voltage — at the cost of extra design complexity (level shifters, voltage domains)

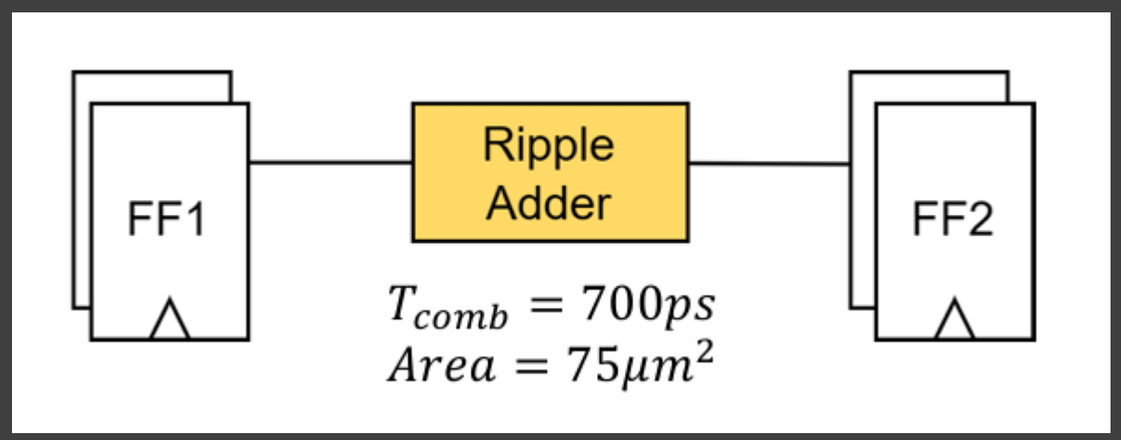
Fixing Timing Violations

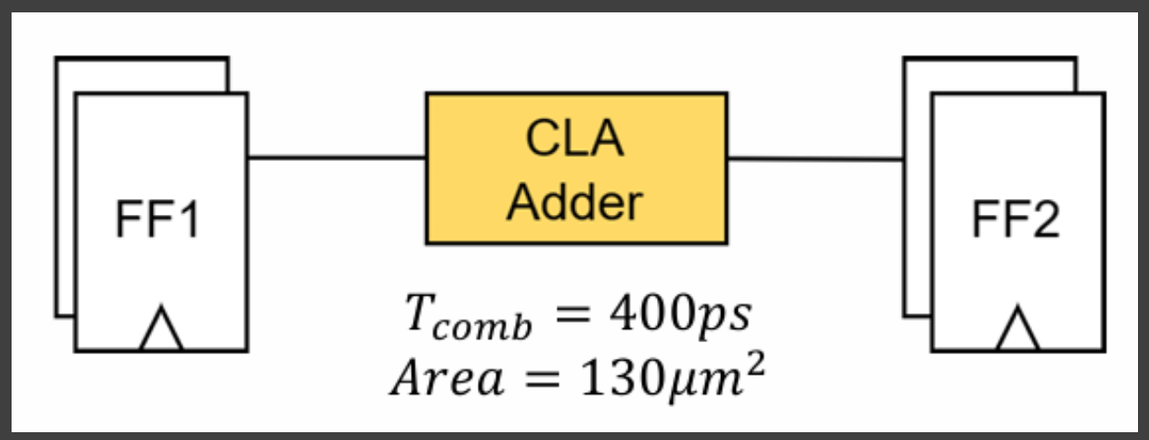
Sol. 4 — Changing the Architecture

- Every digital block has a speed vs. power/area tradeoff — a different implementation can meet timing at a different cost

Example: binary adders

- A ripple-carry adder has small area and power but a long T_{comb} (~700ps)
- A carry-look-ahead (CLA) adder has a shorter T_{comb} (~400ps) but a larger area (130μm^2 vs 75μm^2)
- This decision typically belongs to the RTL/architecture team, since it reshapes the logic itself rather than just re-optimizing it





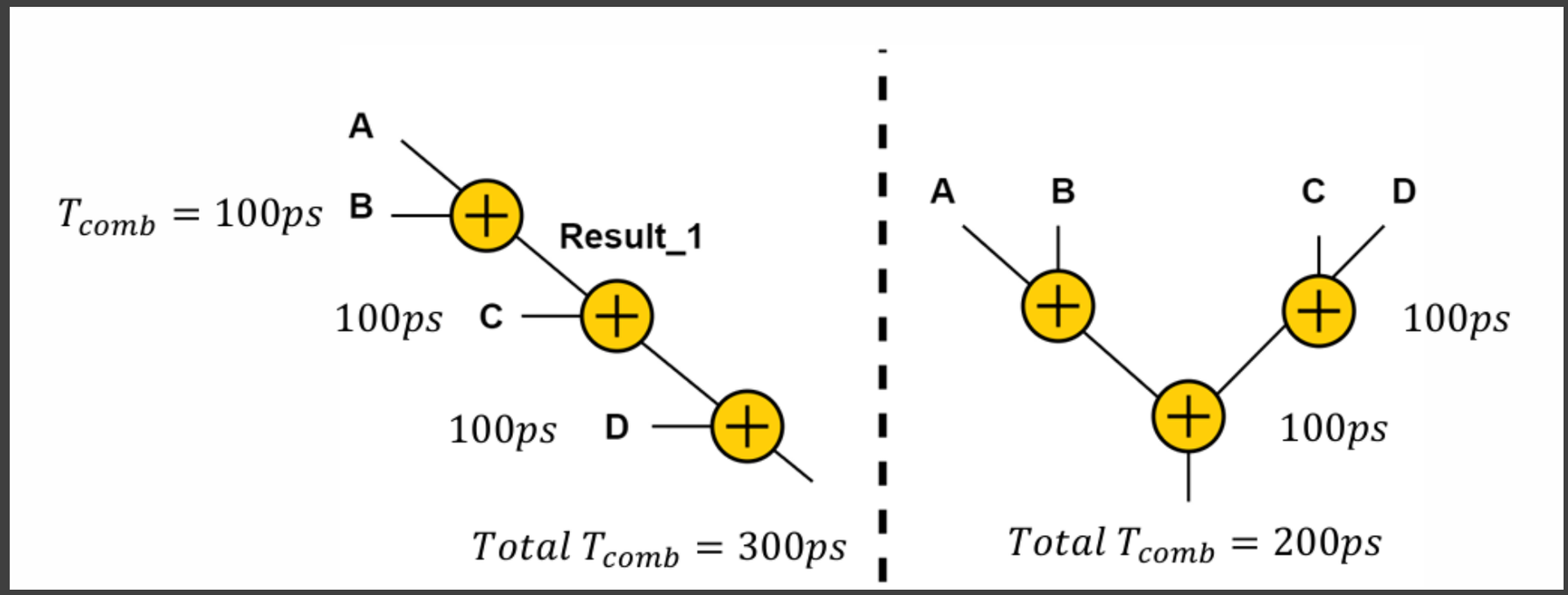
Fixing Timing Violations

Sol. 5 — Optimizing the RTL Code

- The way RTL is written directly affects the resulting gate structure and its delay

Example: computing the same function as a chain (series) of operators vs. a balanced tree

- Chain (series) structure: 3 adders in series → total T_{comb} = 300ps
- Tree (parallel) structure: only 2 adders in series → total T_{comb} = 200ps
- Both circuits implement the same functionality — only the structure of the logic changed



Fixing Timing Violations

Sol. 6 — Pipelining

- The most common RTL-level fix: split a large T_{comb} across multiple clock cycles by inserting pipeline registers
- Example: for A + B*C, do the multiplication in one cycle and the addition in the next, instead of both in one cycle

Trade-offs:

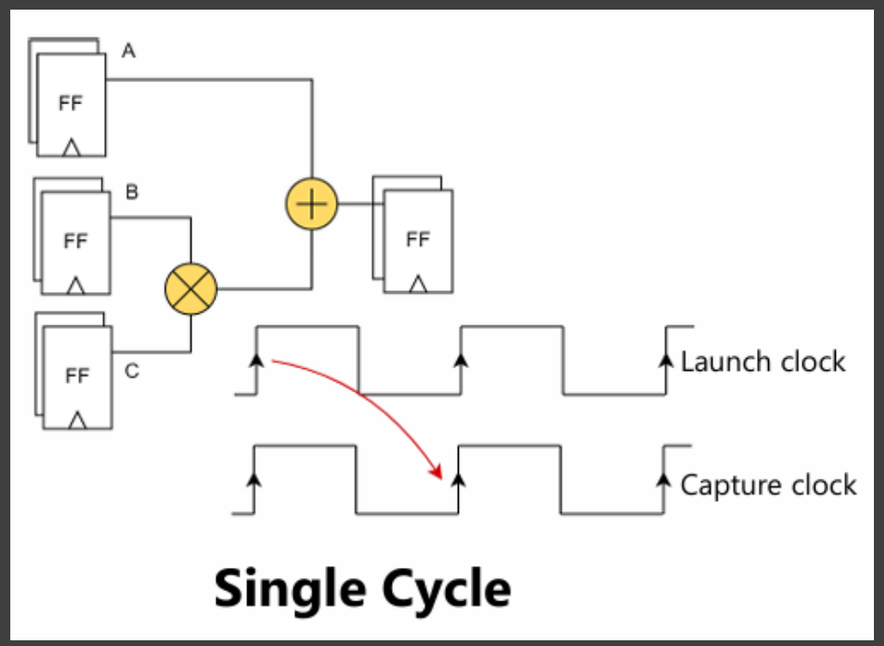
- More area, due to the added pipeline registers
- More latency — the result now takes multiple cycles instead of one
- Synchronization — every input that feeds the delayed stage must also be delayed to match (e.g. A must be pipelined too, or it will get combined with the wrong B*C sample)

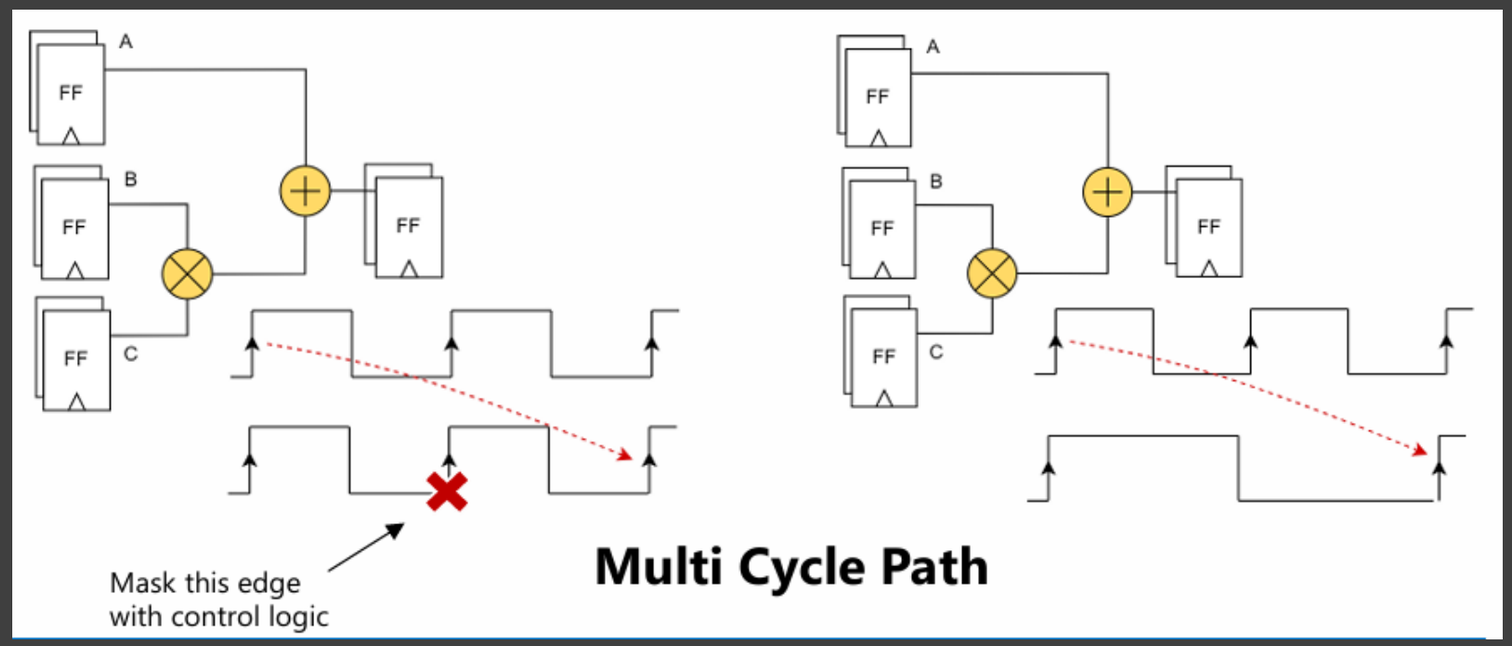
Fixing Timing Violations

Sol. 7 — Multi-Cycle Paths (MCP)

Concept:

- Similar to pipelining — the combinational path is allowed to finish across multiple cycles
- The difference: no pipeline registers are added. Instead, data is captured at a later clock edge
- Done either with a control circuit that masks the first capture edge, or by using a divided clock for the capture FF
- The STA tool must be told about this explicitly using the ***set_multicycle_path*** constraint — it has no knowledge of the circuit's functionality





Fixing Timing Violations

Sol. 7 — Multi-Cycle Paths (MCP)

MCP vs. Pipelining:

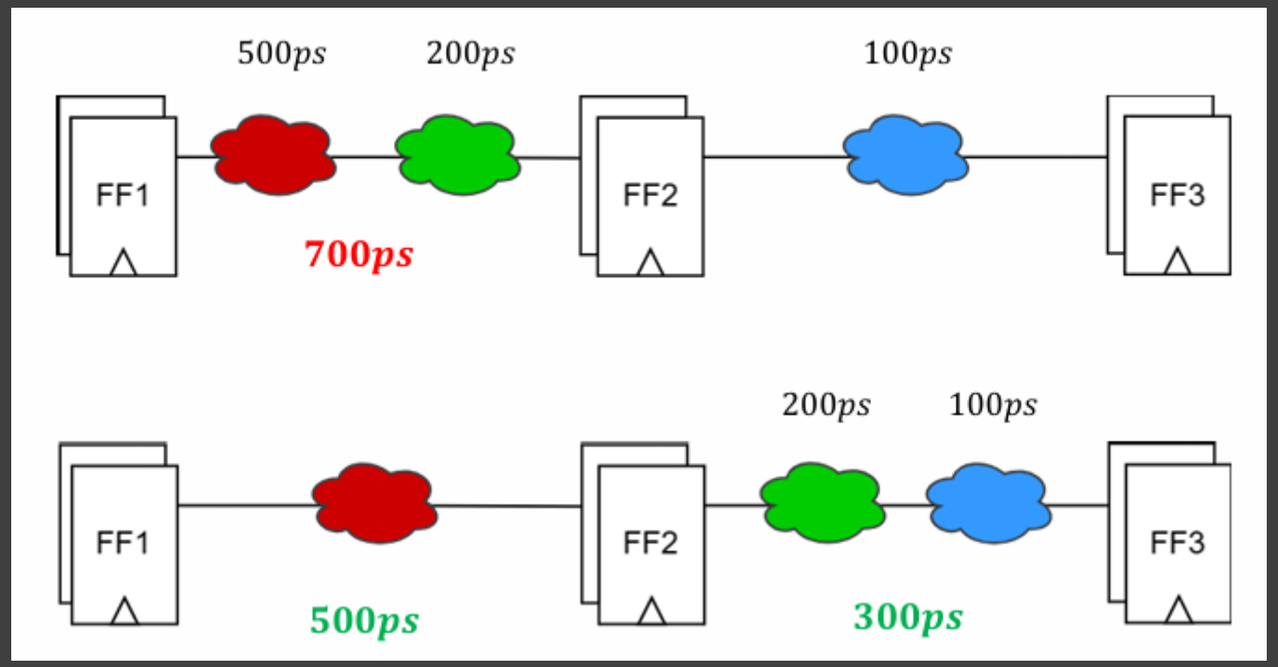
- Pipelining: a new sample enters every cycle — full throughput, extra registers
- MCP: the circuit stays busy for multiple cycles before it can accept a new sample — think of it as reducing the clock frequency, but only for part of the circuit

Fixing Timing Violations

Sol. 8 — Retiming

Concept:

- If T_{comb} is too large to fit in one cycle, split the logic and move part of it into the next pipeline stage
- Example: 700ps of combined logic is split so 500ps stays in the first stage and 300ps moves to join the second stage — both now pass setup
- Can be automatic (synthesis tools) or manual (RTL designer)

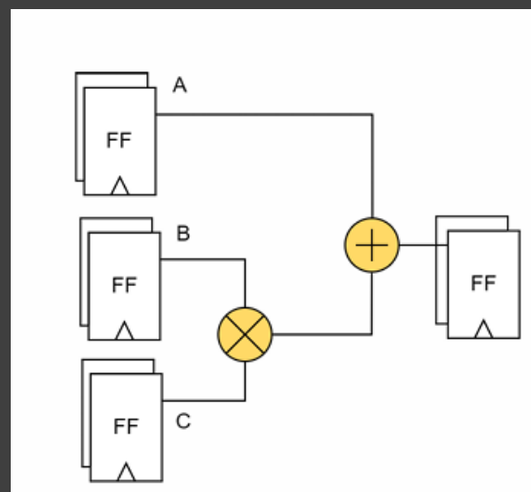


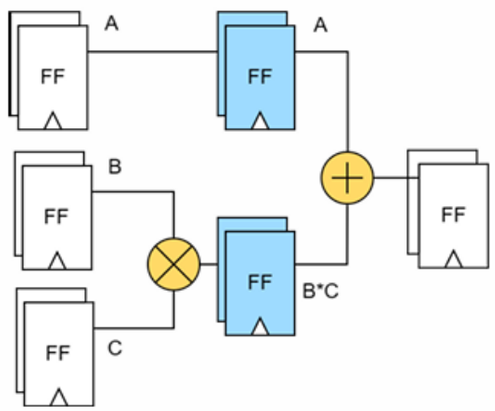
Fixing Timing Violations

Sol. 8 — Retiming

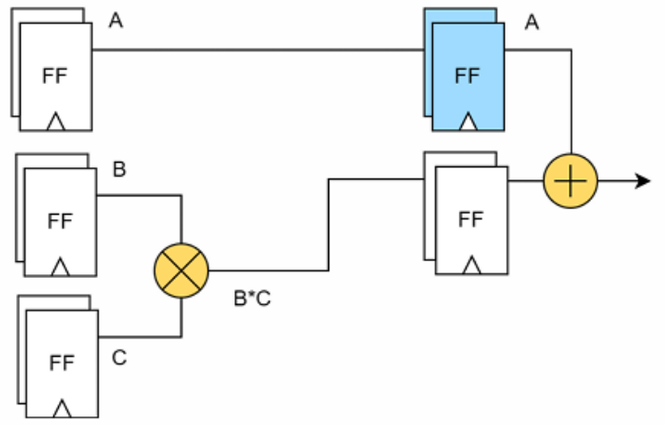
Retiming + Pipelining:

- Moving logic across a register boundary can desynchronize signals that need to arrive together (e.g. A no longer lines up with B*C)
- Synthesis tools will refuse any retiming that breaks functionality this way
- An RTL designer can push further by manually adding a pipeline register to resynchronize the desynced signal — combining retiming with pipelining for extra gain





Pipelining



Pipelining + Retiming

Fixing Timing Violations

Sol. 9 — Optimizing Synthesis

- Synthesis tools expose many switches that trade off the PPA metrics (Power, Performance, Area)

A few examples:

- Increasing timing effort: more optimization, at the cost of runtime
- Reducing or disabling area/power effort: area and power optimizations usually cost timing — dialing them back frees up margin for setup
- Flattening: by default each RTL module is synthesized and kept separate; flattening removes the module boundaries so cells can be optimized across the whole hierarchy, generally improving timing at the cost of verification and debug visibility

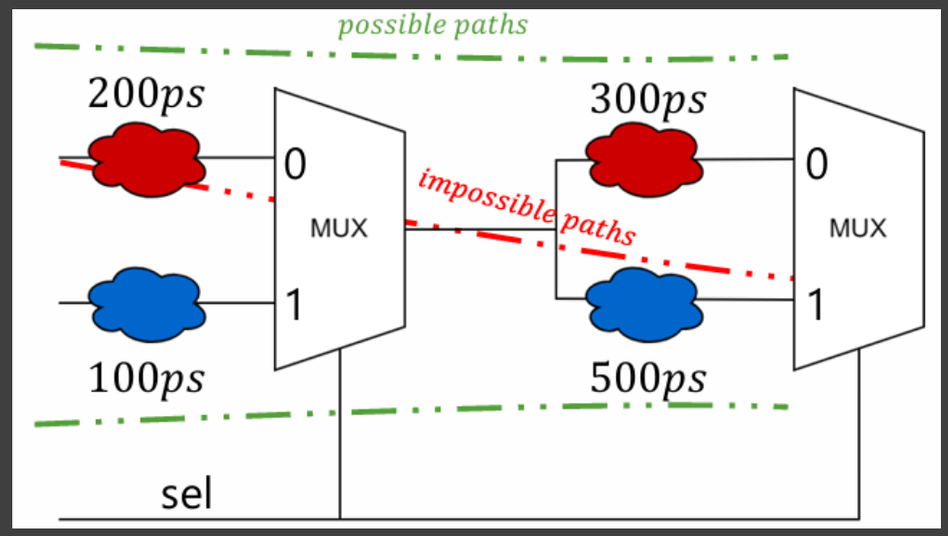
Fixing Timing Violations

Sol. 10 — Applying False Path Constraints

- A false path is a timing path that can never actually occur, given the logic of the circuit

Example:

two muxes share the same select signal — only 2 of the 4 possible input-to-output combinations can ever happen; the other 2 are false paths



- Left unconstrained, the STA tool will analyze these impossible paths anyway — creating fake setup violations
- Worse, this steals optimization effort from the tool: synthesis/PNR apply their most aggressive fixes on the worst paths first, so a fake critical path can starve a real one of attention
- Fix: mark these as false paths in the SDC constraints (***set_false_path***) so the tool ignores them

Fixing Timing Violations

Sol. 11 — Optimizing the Floorplan

- Floorplanning is the first PNR step: chip size/boundaries, placement of major blocks (SRAM, analog), and I/O port placement

What affects setup:

- **Chip area:** a smaller area brings cells closer together (less wire delay), but too small causes voltage drop, congestion, routing detours and crosstalk
- **Block placement:** placing large macros (e.g. SRAMs) near the I/O ports can block standard cells and routing, forcing long detours and longer wire delay
- **Port placement:** poorly placed ports can force long wires and extra buffering, worsening T_{comb}

Fixing Timing Violations

Sol. 12 & 13 — Wire Delay & Power Grid

Optimizing wire delay ($R = \frac{\rho L}{A}, C = \frac{\varepsilon A}{d}$):

- Reduce resistance: shorten the wire (better floorplan) or widen it — higher metal layers are wider and thicker, so critical nets are routed there, manually or via non-default routing rules (NDR)
- Reduce capacitance: increase spacing between wires (via NDR), or reduce the distance two wires run in parallel by moving one to another layer

Relaxing the power grid:

- A wide, compact power grid leaves little routing space for signal nets, blocking spacing/width optimizations
- Relaxing it frees up routing resources for timing, but increases power-grid resistance and voltage drop — a direct trade-off the PNR engineer has to balance

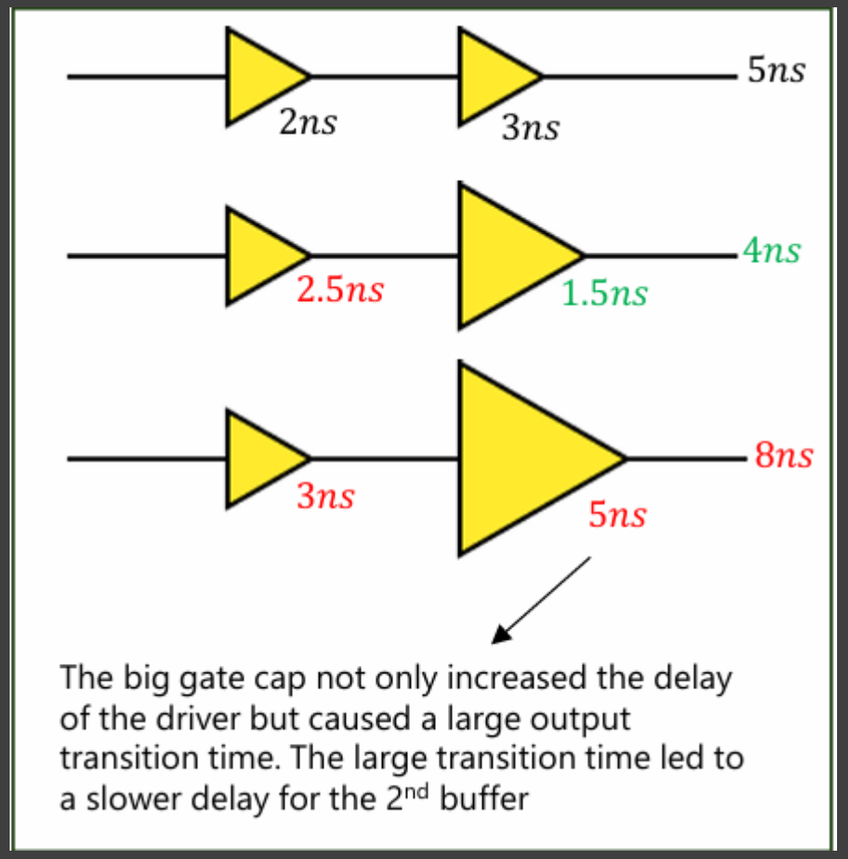
Fixing Timing Violations

Sol. 14 — Upsizing

- Larger MOSFETs drive more current, reducing a cell's propagation delay

Considerations:

- More area and power consumption
- Larger gate capacitance — this slows down whatever cell is driving it, since it now sees a bigger load; the gain from upsizing must outweigh this slowdown of the driver
- Larger cells draw more current, increasing the risk of local voltage drop on neighboring cells
- During ECO, there may not be room for the bigger cell — requiring other cells to be moved and nets rerouted, which can itself affect timing



Fixing Timing Violations

Sol. 15 — MTCMOS / Multi-Vth

- Lowering a cell's threshold voltage (Vth) reduces its propagation delay — at the cost of higher leakage power
- Synthesis and PNR tools let you set a limit on the percentage of low-Vth cells allowed in the design; relaxing that limit improves overall timing
- The gain from swapping Vth is usually smaller than upsizing, but the cell's physical size and location don't change — so no rerouting is needed
- This is why changing cell flavor (Vth swap) is usually the PNR engineer's first go-to fix during ECO
- **Caution:** relaxing the low-Vth limit too freely lets the tool lean on it as an easy fix and skip real logic/wire optimizations, quietly ballooning leakage power

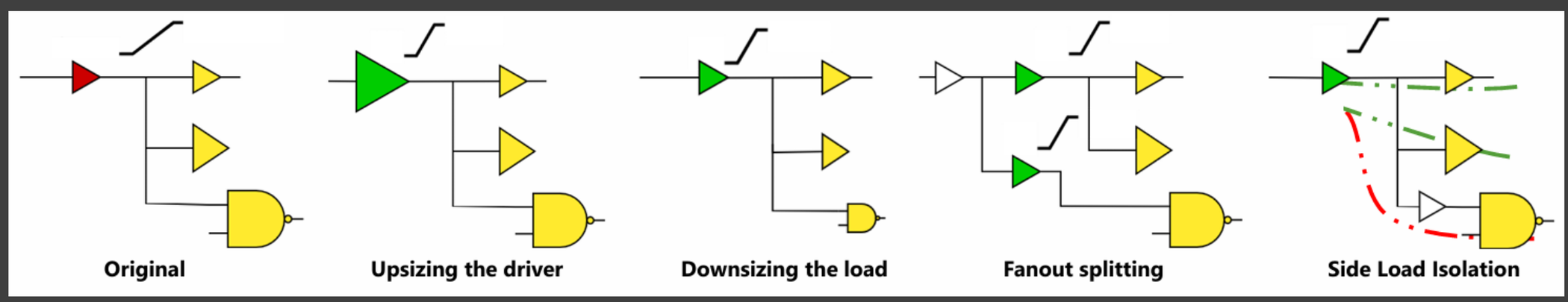
Fixing Timing Violations

Sol. 16 — Increasing Driving Strength

- A cell driving a large load has a slower output transition, which in turn slows down every cell it feeds — improving the driver's strength fixes this at the source

Ways to do it:

- **Upsizing the driver:** more current, faster transition — helps both the driver and everything it feeds
- **Downsizing the load:** less capacitance to drive — but smaller cells are inherently slower, so the net gain must outweigh that
- **Fan-out splitting:** duplicate the driver and split the fan-out between the copies — but the cell driving the driver now sees double the load
- **Side-load isolation:** insert a small buffer to isolate a large load — fixes the isolated path but adds delay to the path through the buffer, so that path needs spare margin



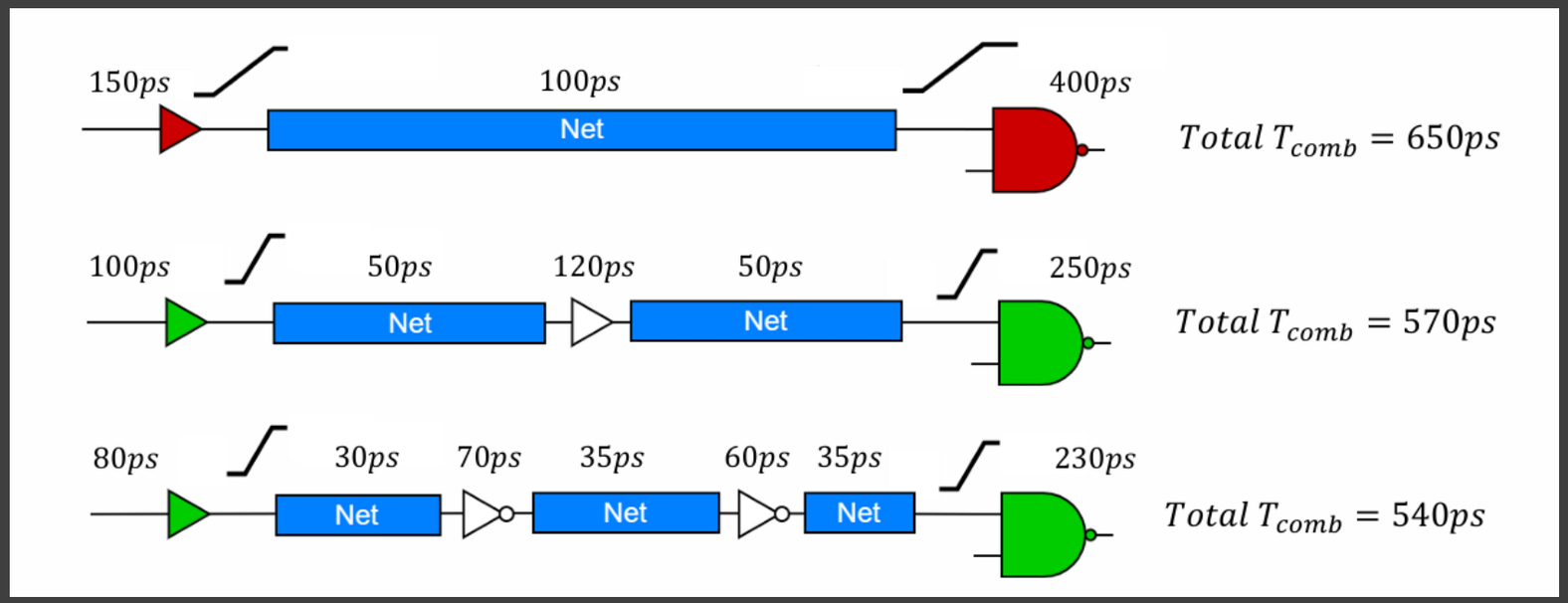
Fixing Timing Violations

Sol. 17 — Breaking Up Long Nets

- A cell driving a very long, high-capacitance wire suffers both slow propagation and slow transition
- Splitting the wire with buffers along its length reduces the load each segment's driver sees
- For very long wires, inverter pairs are often used instead of buffers — an inverter is faster than a buffer of the same size, so you get more cut points for roughly the same total delay

Example:

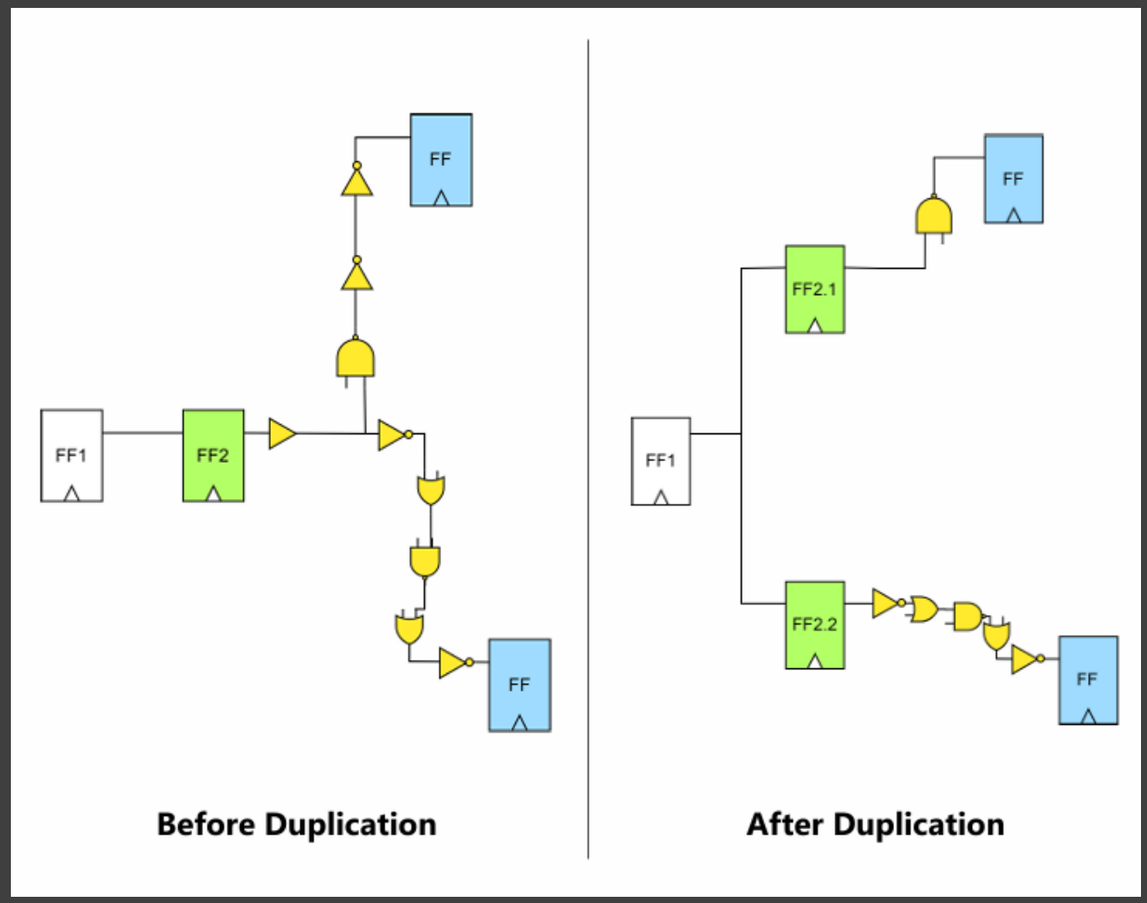
a single long net at 650ps of total delay can drop to ~540ps once split into several buffered segments



Fixing Timing Violations

Sol. 18 — Register Duplication

- Duplicate the launch register and place each copy physically near one of its capture registers
- This shortens the wire between launch and capture, and can remove buffers/inverter pairs that were only there to drive the long net — reducing total combinational delay
- Most useful when the capture registers are placed far apart on the chip
- Trade-off: the original register's driver now feeds double the fan-out, so the path feeding the duplicated registers must be checked for its own setup margin
- Can be done manually in RTL or automatically by synthesis/PNR tools



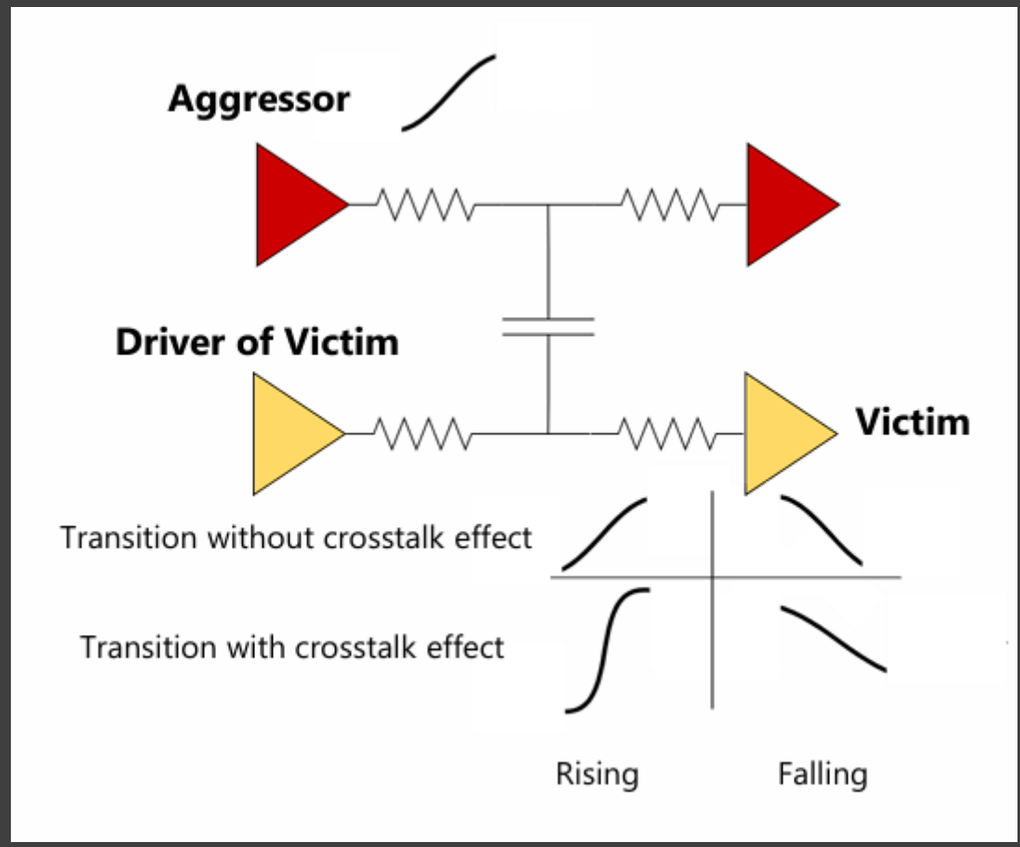
Fixing Timing Violations

Sol. 19 — Reducing Crosstalk

- Coupling capacitance exists between any two adjacent wires — when one (the aggressor) switches, it can speed up or slow down the transition of its neighbor (the victim), depending on whether they switch with the same or opposite polarity
- Opposite-polarity switching slows the victim's transition, increasing T_{comb} along that path

Ways to reduce it:

- Increase spacing between the wires — reduces coupling capacitance directly
- Shield the victim net with VSS wires
- Downsize the aggressor cell to weaken its influence
- Upsize the victim's driver so it can overcome the aggressor's effect — a stronger driver is inherently less sensitive to crosstalk



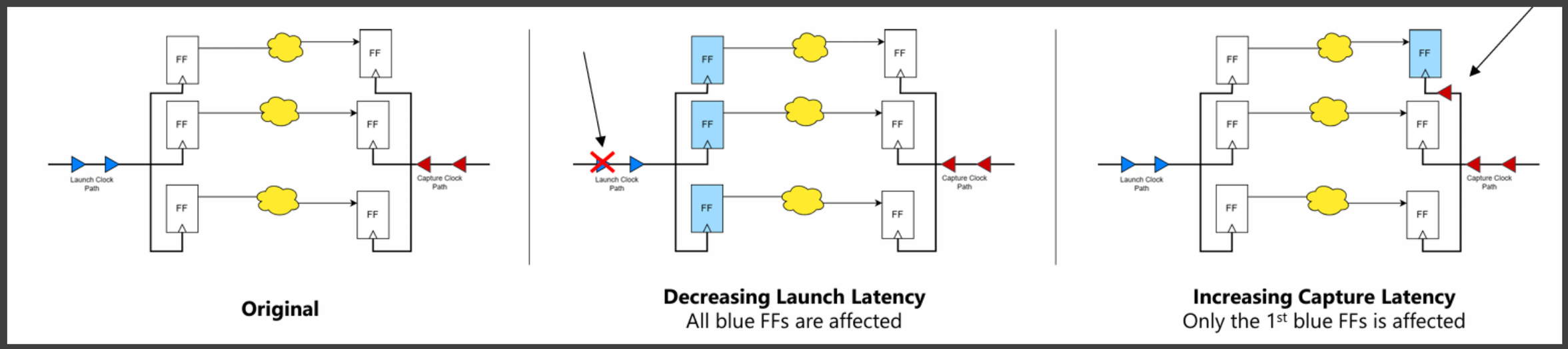
Fixing Timing Violations

Sol. 20 — Local Skew (Borrowing Slack)

- From the setup equation: increasing positive local skew relax setup

How it's done:

- Decrease the clock delay to the launch FF so the clock will reach there sooner by the previous methods.
- Increase the clock delay to the capture FF using buffers to make the clock reach there later.
- In general, increasing the delay is a lot easier than decreasing it because we can simply add buffers.



Fixing Timing Violations

Sol. 20 — Local Skew (Borrowing Slack)

Why it's tricky:

- The launch FF of the current path is the capture FF of the previous path, and the capture FF of the current path is the launch FF of the next path
- Changing skew to fix one path effectively borrows slack from its neighbors — before doing this, the adjacent paths must be checked for enough margin to absorb it
- This method may cause hold violations as the positive skew is good for setup and bad for hold timings

Fixing Timing Violations

How to Fix a Hold Violation

- Comparing the setup and hold equations shows that fixing hold requires the opposite of the setup techniques

In practice:

- Instead of decreasing T_{comb}, we increase it — inserting buffers, using inverter pairs, downsizing cells, or lengthening wires
- Instead of increasing positive local skews, we do the opposite — increase negative local skews

Why this matters:

- Hold fixes tend to work against setup fixes on the same path — this is why increasing delay (adding buffers) is the go-to hold fix: it's easier than decreasing delay, and generally doesn't require re-optimizing the whole path the way a setup fix does

Fixing Timing Violations

Setup vs. Hold — Why Setup Has Priority

- Increasing delay (buffers) is always easier than decreasing it — this makes hold generally easier to fix than setup
- Because of this, setup is given priority: hold-fixing effort starts only after all setup violations are resolved
- Hold is still monitored throughout the PNR stages to make sure it stays fixable, even while the main focus is on closing setup
- This is also why hold is fixed almost exclusively during the PNR/ECO stage — RTL and synthesis focus on setup

Fixing Timing Violations

Summary — Key Takeaways

- Setup violations are fixed by decreasing the arrival time or increasing the required time — from architecture-level choices (frequency, node, voltage) down to cell-level ECO fixes (upsizing, Vt swap, buffering)
- Hold violations are fixed with largely the opposite techniques — mainly by adding delay, since increasing delay is easier than decreasing it
- Earlier design stages can fix the largest violations; later stages (ECO) are limited to small, surgical, and often manual fixes
- Setup is prioritized over hold, because hold is comparatively easier to close once setup is stable
- Every fix is a trade-off against area, power, or another timing path — nothing is free, which is why understanding the full picture of the timing path matters as much as knowing the individual techniques

Timing Constraints & SDC

Timing Constraints & SDC

What is SDC?

Synopsys Design Constraints (SDC):

- Industry-standard format for timing constraints
- Used by all major EDA tools (PrimeTime, Design Compiler, Vivado)
- Tcl-based commands for specifying timing requirements

Key SDC Components:

- Clock definitions
- Input/output delay constraints
- Timing exceptions (false paths, multicycle paths)
- Driving and load constraints
- Uncertainty specifications

Timing Constraints & SDC

Clock Definition

Basic Clock Definition:

verilog-tut - Basic_Clock_Creation.tcl

```tcl
### Basic Clock Creation

### Syntax
create_clock -name <clock_name> -period <period> [get_ports <port_name>]
```

Example:

verilog-tut - Basic_Clock_Creation.tcl

```tcl
### Examples
### Default Waveform
# 100 MHz clock (10ns period)
create_clock -name sys_clk -period 10.0 [get_ports clk]
```

This Creates:

- A clock named ‘sys_clk'
- Period of 10ns (100MHz frequency)
- Rising edge at time 0, falling at 5ns
- 50% duty cycle by default

Timing Constraints & SDC

Clock Definition

Examples:

verilog-tut - Basic_Clock_Creation.tcl

```tcl
### Custom Waveform (Non-50% Duty Cycle)
# 40% duty cycle: high for 4ns, low for 6ns
create_clock -name clk_40 -period 10.0 -waveform {0 4} [get_ports clk]

# 30% duty cycle: high for 3ns, low for 7ns
create_clock -name clk_30 -period 10.0 -waveform {0 3} [get_ports clk]

### Inverted Clock (Starts Low)
# Starts low, rises at 5ns, falls at 10ns (one period later)
create_clock -name clk_inv -period 10.0 -waveform {5 10} [get_ports clk_n]

### Multiple Independent Clocks

### Asynchronous Clocks
# Fast clock for processor
create_clock -name cpu_clk -period 2.0 [get_ports clk_cpu]

# Slow clock for peripherals
create_clock -name peri_clk -period 50.0 [get_ports clk_peri]

# Declare them asynchronous (no timing relationship)
set_clock_groups -asynchronous \
    -group {cpu_clk} \
    -group {peri_clk}
```

- Without ***set_clock_groups***, the tool might try to analyze paths between these unrelated clocks.

Timing Constraints & SDC

Generated Clocks

verilog-tut - Basic_Clock_Creation.tcl

```tcl
### Generated Clocks
### Clock Divider
# Primary clock
create_clock -name main_clk -period 10.0 [get_ports clk]

# Divided-by-2 clock
create_generated_clock -name div2_clk \
    -source [get_ports clk] \
    -divide_by 2 \
    [get_pins divider/Q]

### Clock Multiplier (PLL)
# Input reference clock
create_clock -name ref_clk -period 20.0 [get_ports ref_clk]

# PLL output (multiply by 4)
create_generated_clock -name pll_clk \
    -source [get_ports ref_clk] \
    -multiply_by 4 \
    [get_pins pll/clk_out]
```

Timing Constraints & SDC

Clock Properties

verilog-tut - Basic_Clock_Creation.tcl

```tcl
### Clock Properties
### Clock Uncertainty
# Jitter and skew
set_clock_uncertainty -setup 0.2 [get_clocks sys_clk]
set_clock_uncertainty -hold 0.1 [get_clocks sys_clk]

### Clock Latency
# Source latency (before clock tree)
set_clock_latency -source 0.5 [get_clocks sys_clk]

# Network latency (clock tree delay)
set_clock_latency 1.0 [get_clocks sys_clk]

### Clock Transition
# Clock edge slew rate
set_clock_transition 0.1 [get_clocks sys_clk]
```

Timing Constraints & SDC

Virtual Clocks

Use Case: I/O timing when external clock isn't in your design

verilog-tut - Basic_Clock_Creation.tcl

```tcl
### Virtual Clocks
# Virtual clock for external system
create_clock -name ext_sys_clk -period 8.0

# Input timing relative to virtual clock
set_input_delay -clock ext_sys_clk -max 2.0 [get_ports data_in]

# Output timing relative to virtual clock
set_output_delay -clock ext_sys_clk -max 1.5 [get_ports data_out]
```

- The virtual clock exists only for constraint purposes—there's no physical pin.

Timing Constraints & SDC

Input and Output Delays

- Your chip interfaces with the outside world—other chips, memories, sensors. I/O delays model the timing relationship between your internal clock and external signals.

- We can model the input and output delays for better analysis.

Timing Constraints & SDC

Input Delays

- Syntax

verilog-tut - Input_and_Output_Delays.tcl

```tcl
### Syntax
set_input_delay -clock <clock_name> [-max|-min] <delay> [get_ports <port_pattern>]

### Examples
# External device launches data with 3ns delay
set_input_delay -clock sys_clk -max 3.0 [get_ports data_in]

# Minimum delay (for hold check)
set_input_delay -clock sys_clk -min 1.0 [get_ports data_in]
```

- This means:
  - Data arrives 3.0ns after the clock edge (worst case)
  - Data arrives 1.0ns after the clock edge (best case)

Timing Constraints & SDC

Input Delays

- **Examples:**

verilog-tut - Input_and_Output_Delays.tcl

```
### Applying to Multiple Ports
# All input data ports
set_input_delay -clock sys_clk -max 2.5 [get_ports data_in*]

# Specific list of ports
set_input_delay -clock sys_clk -max 2.0 [get_ports {addr[0] addr[1] addr[2]}]

# All inputs except clock
set_input_delay -clock sys_clk -max 2.0 [all_inputs]

### Different Delays for Rising and Falling
# Rising edge
set_input_delay -clock sys_clk -clock_fall -max 2.0 [get_ports data]

# Falling edge
set_input_delay -clock sys_clk -clock_fall -max 2.5 [get_ports data]
```

Timing Constraints & SDC

Output Delays

- Syntax

verilog-tut - Input_and_Output_Delays.tcl

```tcl
### Output Delays

### Syntax
set_output_delay -clock <clock_name> [-max|-min] <delay> [get_ports <port_pattern>]

### Examples
# External device needs data 1.5ns before its clock edge
set_output_delay -clock sys_clk -max 1.5 [get_ports data_out]

# Minimum output delay (for hold)
set_output_delay -clock sys_clk -min 0.3 [get_ports data_out]
```

- This means:
  - Data must be stable 1.5ns before the external device's clock edge
  - Data can change 0.3ns after the external device's clock edge

Timing Constraints & SDC

Add vs. Set: Cumulative Delays

verilog-tut - Input_and_Output_Delays.tcl

```
### Add vs. Set: Cumulative Delays
### Set (Default): Replaces previous value
set_input_delay -clock clk -max 2.0 [get_ports data]
set_input_delay -clock clk -max 3.0 [get_ports data]  # Overrides to 3.0

### Add: Accumulates
set_input_delay -clock clk -max 2.0 [get_ports data]
set_input_delay -clock clk -max -add 1.0 [get_ports data]  # Total = 3.0
```

Timing Constraints & SDC

Load and Drive Specifications

- **Output Load:** Model capacitive load on output ports.

verilog-tut - Load_and_Drive_Specifications.tcl

```tcl
## Load and Drive Specifications
### Output Load
set_load <capacitance> [get_ports <port_pattern>]

### Examples
# 50 fF load (typical board trace + input cap)
set_load 0.05 [get_ports data_out[*]]

# Different loads for different outputs
set_load 0.03 [get_ports control_signals[*]]
set_load 0.08 [get_ports high_cap_outputs[*]]

# Default load for all outputs
set_load 0.05 [all_outputs]

# Units: Typically picofarads (pF) or femtofarads (fF) depending on Liberty file units
```

- Impact on Timing:
  - Larger loads → slower transitions → longer delays
  - Synthesis/STA use this for accurate output delay calculation

Timing Constraints & SDC

Load and Drive Specifications

- **Input Load:** Model the strength of the external driver.

verilog-tut - Load_and_Drive_Specifications.tcl

```tcl
### Input Load
set_driving_cell -lib_cell <cell_name> [get_ports <port_pattern>]

### Examples
# External chip drives with a medium-strength buffer
set_driving_cell -lib_cell BUF_X2 [get_ports data_in[*]]

# Weak driver (high impedance input)
set_driving_cell -lib_cell BUF_X1 [get_ports slow_inputs[*]]

# Strong driver (low impedance)
set_driving_cell -lib_cell BUF_X8 [get_ports fast_inputs[*]]

### Alternative: Direct Transition Time
# Specify input transition directly
set_input_transition 0.2 [get_ports data_in[*]]
```

- What does that does:
  - Uses the Liberty cell's output characteristics
  - Determines input transition time
  - Affects downstream timing calculations

Timing Constraints & SDC

Example

verilog-tut - Load_and_Drive_Specifications.tcl

```tcl
### Load and Drive: Complete I/O Example
# ============================================================
# Input Constraints
# ============================================================
# External CPU drives our inputs with strong buffers
set_driving_cell -lib_cell BUF_X4 [all_inputs]

# Input timing from CPU datasheet
set_input_delay -clock sys_clk -max 3.0 [get_ports data_in[*]]
set_input_delay -clock sys_clk -min 1.0 [get_ports data_in[*]]

# ============================================================
# Output Constraints
# ============================================================
# Drive external SRAM (moderate capacitance)
set_load 0.06 [get_ports mem_addr[*]]
set_load 0.08 [get_ports mem_data[*]]

# Output timing per SRAM datasheet
set_output_delay -clock sys_clk -max 2.5 [get_ports mem_addr[*]]
set_output_delay -clock sys_clk -max 2.5 [get_ports mem_data[*]]
```

Timing Constraints & SDC

Max and Min Delay Constraints

- We can put constrains on certain paths with a certain delay value.

- **set_max_delay**

verilog-tut - Max_and_Min_Delay_Constraints.tcl

```tcl
### Max and Min Delay Constraints
### set_max_delay
set_max_delay <delay> -from <from_list> -to <to_list>

### Use Cases:
### [1] Asynchronous Paths
# Async reset distribution: must arrive within 5ns
set_max_delay 5.0 -from [get_ports async_rst] -to [all_registers]

### [2] Combinational Paths (No Clocks)
# Pure combinational logic from input to output
set_max_delay 3.0 -from [get_ports combo_in] -to [get_ports combo_out]
```

- **set_min_delay**

verilog-tut - Max_and_Min_Delay_Constraints.tcl

```tcl
### set_min_delay
set_min_delay <delay> -from <from_list> -to <to_list>

### Use Cases:
### [1] Hold Margin on Critical Paths
# Ensure at least 0.5ns delay (prevent hold violations)
set_min_delay 0.5 -from [get_ports fast_input] -to [get_registers]
```

Timing Constraints & SDC

False Paths and Multicycle Paths

- **False Paths:** Disable timing analysis on paths that will never propagate data.

verilog-tut - False_Paths_and_Multicycle_Paths.tcl

```tcl
### False Paths and Multicycle Paths
### False Paths
set_false_path -from <from_list> -to <to_list>

### Common Scenarios:
### [1] Configuration/Test Paths
# Config registers don't change during normal operation
set_false_path -from [get_ports config_mode] -to [all_registers]
# Test scan chain
set_false_path -from [get_ports scan_enable] -to [all_registers]

### [2] Through Specific Pins
# Ignore paths through debug multiplexer
set_false_path -through [get_pins debug_mux/debug_select]

### [3] Reset Paths
# Async reset doesn't need timing check
set_false_path -from [get_ports rst_n] -to [all_registers]

### [4] From/To/Through combination
set_false_path -from [get_clocks clk_a] \
               -through [get_pins sync_ff/Q] \
               -to [get_clocks clk_b]
```

Timing Constraints & SDC

False Paths and Multicycle Paths

- **Multicycle Paths:** Relax timing requirements for paths that take multiple clock cycles

verilog-tut - False_Paths_and_Multicycle_Paths.tcl

```tcl
### Multicycle Paths
set_multicycle_path <num_cycles> -setup|-hold -from <from> -to <to>

### Common Scenarios:
### [1] Slow Functional Units
# 32-cycle integer divider
set_multicycle_path 32 -setup -from [get_pins divider/*] \
                               -to [get_pins result_reg/D]
set_multicycle_path 31 -hold  -from [get_pins divider/*] \
                               -to [get_pins result_reg/D]

### [2] Multi-Cycle State Machine
# State machine takes 4 cycles per state
set_multicycle_path 4 -setup -from [get_pins state_reg[*]/Q] \
                              -to [get_pins state_reg[*]/D]
set_multicycle_path 3 -hold  -from [get_pins state_reg[*]/Q] \
                              -to [get_pins state_reg[*]/D]

### [3] Data Valid Enable Signal
# Data updates every 3 cycles (controlled by enable)
set_multicycle_path 3 -setup -from [get_pins data_reg[*]/Q]
set_multicycle_path 2 -hold  -from [get_pins data_reg[*]/Q]

### Example
# Path can take 2 clock cycles
set_multicycle_path 2 -setup \
    -from [get_pins launch_ff/Q] \
    -to [get_pins capture_ff/D]

# Adjust hold check (1 cycle, not 2)
set_multicycle_path 1 -hold \
    -from [get_pins launch_ff/Q] \
    -to [get_pins capture_ff/D]
```

Timing Constraints & SDC

False Paths and Multicycle Paths

- **Why Hold is N-1?**
  - Setup checks at cycle N (e.g., cycle 2)
  - Hold checks at cycle N-1 (e.g., cycle 1)
  - Prevents overly relaxed hold check

- STA tools will by default, select the hold capture edge to be the edge one cycle before the setup edge.

- We fix this by instructing the tool to set hold N-1 cycles of the setup edge, where N is the number of multi-cycles.

- Always specify both setup and hold for multicycle paths.

Advanced Timing Concepts

Advanced Timing Concepts

On-Chip Variation

On-Chip Variation (OCV):

- Process, voltage, and temperature variations across the die
- Different parts of chip experience different conditions
- More significant at advanced technology nodes (7nm, 5nm, 3nm)

Types of OCV:

- **Global variation:** affects entire chip similarly
- **Local variation:** affects neighboring transistors differently
- **Systematic variation:** pattern-dependent (layout effects)

Impact:

Path delays can vary significantly from nominal characterization

Advanced Timing Concepts

Clock Reconvergence Pessimism

Clock Reconvergence Pessimism Removal (CRPR):

- Accounts for overly pessimistic timing analysis
- Launch and capture clock paths share common portion
- Default STA assumes worst-case variation in both paths
- CRPR recognizes they vary together (not independently)

How It Works:

- Removes pessimism from common clock path
- Can recover 100-200ps of timing margin in advanced nodes
- Essential for realistic signoff timing

Interview Question:

What is CRPR and why is it needed?

Advanced Timing Concepts

Timing Derates

Timing Derates:

- Scaling factors to account for PVT variations
- Applied to delays during analysis

Setup Analysis (Slow Corner):

- Data path delays scaled UP (worst-case propagation)
- Clock delays scaled DOWN (worst-case for setup)

Hold Analysis (Fast Corner):

- Data path delays scaled DOWN (fast propagation)
- Clock delays scaled UP (worst-case for hold)

Advanced Derating (AOCV):

Path-based derating depending on path depth

Advanced Timing Concepts

Signal Integrity & Crosstalk

Signal Integrity (SI) Impact on Timing:

- Crosstalk from neighboring nets can delay or speed signals
- Aggressor net switching affects victim net timing
- Must be analyzed for signoff at advanced nodes

Crosstalk Effects:

- **Delay pushout:** Victim slows down (worse for setup)
- **Speedup:** Victim speeds up (worse for hold)

Analysis Methods:

- Vectorless analysis (no patterns needed)
- Pattern-based analysis (using switching activity)
- Primetime SI tool for crosstalk-aware STA

Signoff & Corners

Signoff & Corners

PVT Operating Corners

Process, Voltage, Temperature (PVT) Corners:

- Design must work across all operating conditions
- Multiple corners analyzed for comprehensive verification

Process Corners:

- **SS (Slow-Slow):** Transistors slow, worst for setup
- **FF (Fast-Fast):** Transistors fast, worst for hold
- **TT (Typical-Typical):** Nominal case
- SF, FS: Mixed corners for specific analysis

Voltage & Temperature:

- **Low voltage:** slower transistors (worst for timing)
- **High temperature:** slower for most cells (negative temp coefficient)

Signoff & Corners

Signoff Corners Explained

Typical Signoff Corners:

- **Setup:** SS process, low voltage, high temperature
- **Hold:** FF process, high voltage, low temperature
- **Additional:** Nominal corner for initial verification

Process Variations:

- Channel length variation
- Threshold voltage variation
- Mobility variation

Why Multiple Corners:

- Designs must work in all customer environments
- **Automotive:** -40°C to 150°C temperature range
- **Consumer:** 0°C to 85°C typical

References

REFERENCES

- "Static Timing Analysis" Series of articles on LinkedIn by Eng. Amr Adel.
- "Timing analysis session" IEEE ASU
- What is Static Timing Analysis (STA)?
- clock-skew-implication-on-timing blog
- RC Charging Circuit Tutorial & RC Time Constant
- Chip verify: SDC section

FRIENDLY REMINDER

Israel has killed more than 13,000 children in Gaza since October 7 while others are suffering from severe malnutrition and do not “even have the energy to cry”,
says the United Nations Children’s Fund (UNICEF).

“Thousands more have been injured or we can’t even determine where they are. They may be stuck under rubble … We haven’t seen that rate of death among children in almost any other conflict in the world”
-UNICEF Executive Director

Save The Palestinian Children

THANKS